

# Cloud Governance Agent System

## Design Specification

A hierarchical multi-agent recommender system for cloud resource governance.  
Four agents, four engineering harnesses, replayable audit trail, human-in-the-loop review.

Version 1  
May 2026

Companion to the Cloud Governance Dataset Generation Plan v3

# Table of Contents

## 1. Architecture

- 1.1 The system at a glance
- 1.2 The four-harness layering
- 1.3 End-to-end flow of a single review
- 1.4 Tier specialist internals — the ReAct loop
- 1.5 Cross-Tier Evaluator internals — drift-check, then synthesize
- 1.6 Where each harness shows up in the flow
- 1.7 What this architecture is not

## 2. Design Principles

- 1.1 Recommender, not actuator
- 1.2 Hierarchical multi-agent — earned, not decorative
- 1.3 Failure modes are about quality, consistency, and accountability
- 1.4 Synthetic data, deliberately
- 1.5 The harnesses provide properties, not defenses
- 1.6 Specialization over scale
- 1.7 The trade-offs are part of the deliverable

## 3. Agents

- 2.1 Supervisor
- 2.2 System Mapper
- 2.3 Tier Specialists (×3)
- 2.4 Cross-Tier Evaluator
- 2.5 Why this specific agent set

## 4. Harnesses

- 3.1 The four harnesses at a glance
- 3.2 Input Harness
- 3.3 Reasoning Harness
- 3.4 Action Harness
- 3.5 Persistent Action Record
- 3.6 How the harnesses interact

## 5. ReAct Tool Surface

- 4.1 The read contract, as agent tool surface
- 4.2 Per-agent tool surfaces
- 4.3 Why structural scoping matters
- 4.4 The ReAct loop, in terms of this surface
- 4.5 What the tool surface is not

## 6. Confidence and Drift

- 5.1 Two levels of confidence

- 5.2 Why this split matters
- 5.3 The Evaluator's drift-check
- 5.4 Policy choices that need to be tuned during build
- 5.5 What drift-check does not catch
- 5.6 How this shows up in the review packet

## **7. Persistent Action Record**

- 6.1 What the audit trail captures
- 6.2 Append-only by design
- 6.3 Replayability
- 6.4 Why relational, not vector
- 6.5 Conceptual schema
- 6.6 What the audit trail does not do
- 6.7 The audit trail as the README's visual hero

## **8. Evaluation Methodology**

- 7.1 Two evaluation layers, distinct concerns
- 7.2 The agent evaluation rubric (R1–R5)
- 7.3 LLM-as-judge for the rubric
- 7.4 Per-scenario scoring and aggregate scoring
- 7.5 Tolerance bands for numeric matching
- 7.6 The three subset modes
- 7.7 What the evaluation methodology does not do
- 7.8 The eval set as a build artifact

## **9. LLM Provider Mix and Cost**

- 8.1 The two-tier model split
- 8.2 Why not a single model throughout
- 8.3 Cost estimation per review
- 8.4 Cost-aware design patterns
- 8.5 The eval cost is separate from the system cost
- 8.6 What if the budget is tighter
- 8.7 Provider portability

## **10. Design Decisions and Trade-offs**

- 9.1 ReAct over zero-shot
- 9.2 Relational over vector for the audit trail
- 9.3 Action Harness intentionally narrow
- 9.4 Two-tier LLM mix
- 9.5 Terraform parsing for the System Mapper
- 9.6 Three specialists in parallel
- 9.7 Two-level confidence model
- 9.8 Hand-crafted synthetic dataset
- 9.9 Recommender-only, no actuation

9.10 What I deliberately said no to

9.11 The shape of this section in an interview

## **11. Limitations**

10.1 Identity-driven limits

10.2 Scope-driven limits

10.3 Dataset-driven limits

10.4 What this is not

10.5 Where the limits would relax in a production-shaped extension

10.6 The point of this document

## **12. Build Plan**

Phase structure

Phase 1 — Audit trail and ingest

Phase 2 — Orchestration and structure

Phase 3 — Reasoning depth

Phase 4 — Synthesis, gating, release

Cross-cutting concerns

What ships at the end of Phase 4

# 1. Architecture

This document presents the agent topology, the harness layering, and the end-to-end flow of a single review cycle. It is the structural counterpart to the prose descriptions in `docs/02-agents.md` (agents) and `docs/03-harnesses.md` (harnesses).

## 1. The system at a glance

Two architectural concerns sit perpendicular to each other:

- **The agent topology** — who reasons about what, and in what order.
- **The harness layering** — what structure, safety, and observability properties are enforced across all agents.

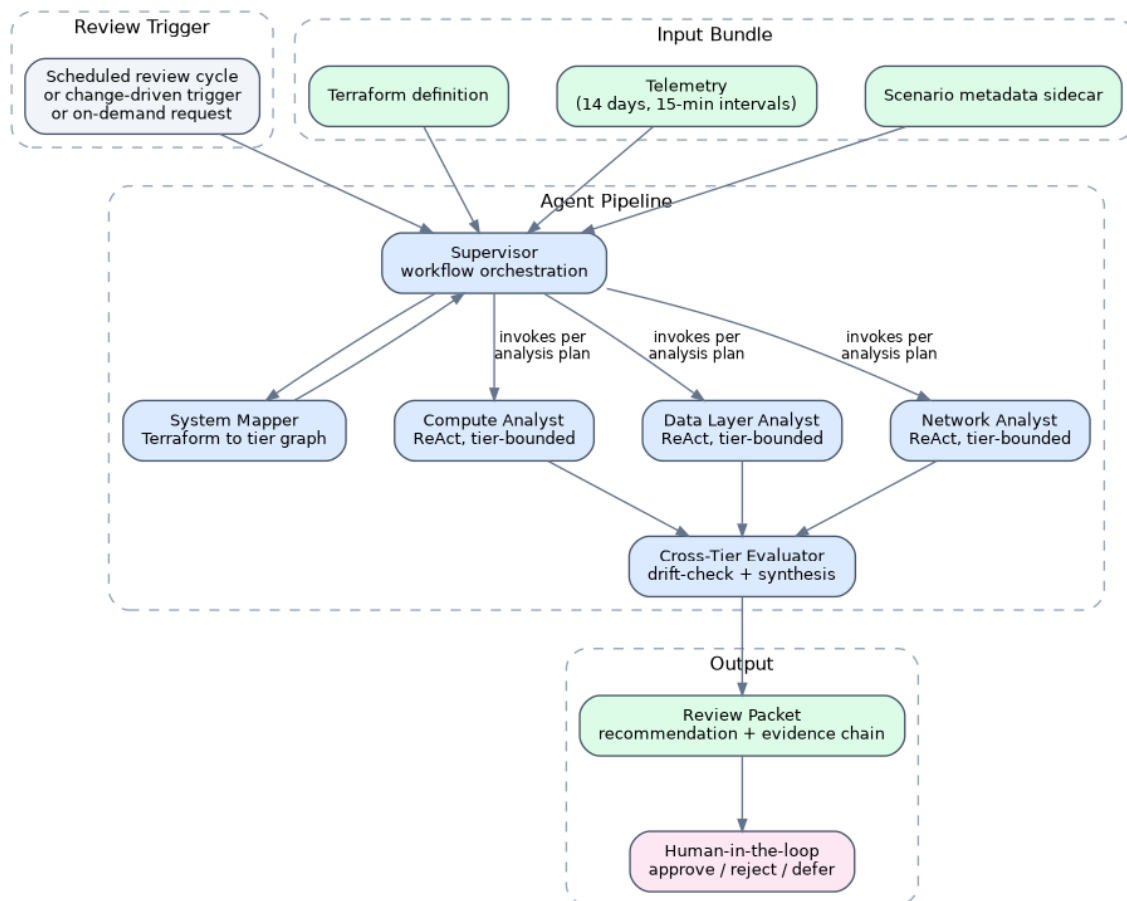


Figure 1: System overview — agent topology with input and output flow

Every arrow in the diagram crosses one or more harnesses. The next section describes the harnesses that sit underneath this topology.

## 2. The four-harness layering

The harnesses are not a fifth agent. They are properties enforced across the agents and across the data that flows between them.

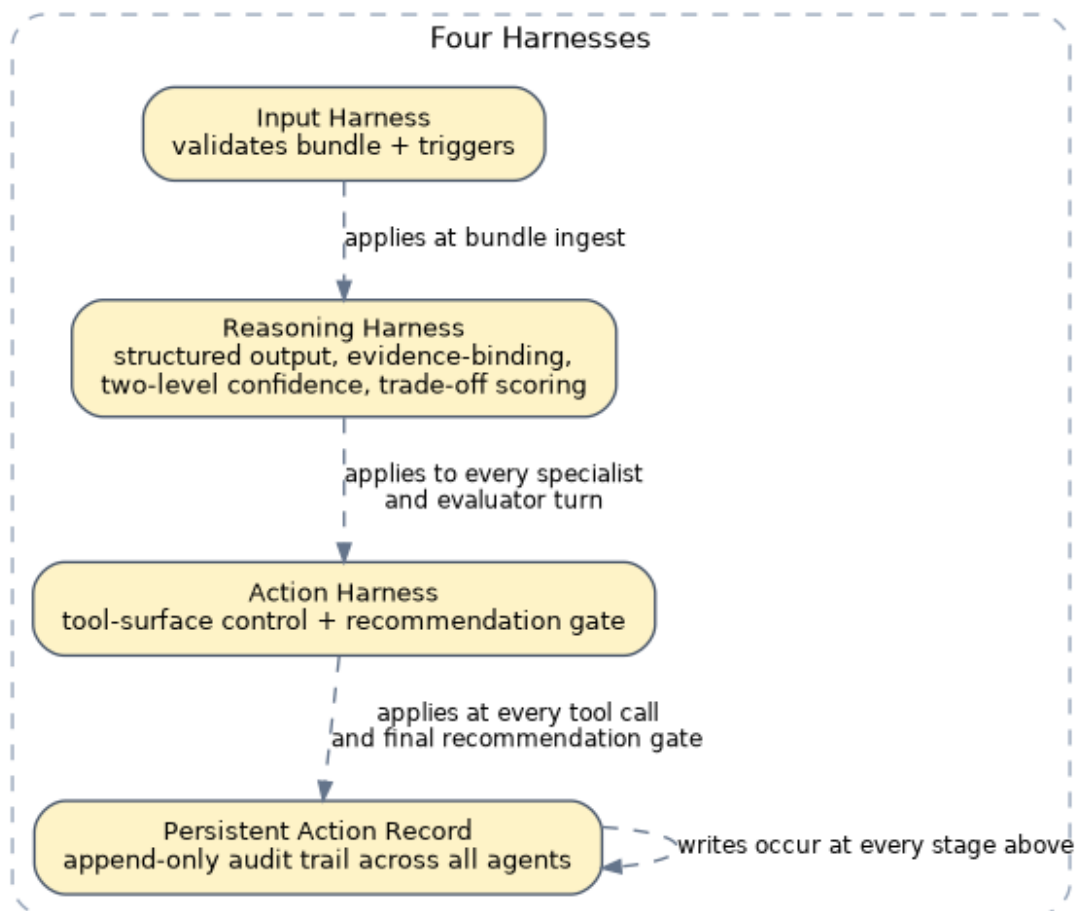


Figure 2: The four harnesses as cross-cutting properties

The Persistent Action Record is the only harness with its own dedicated storage. The other three are validation and shaping layers that run alongside agent execution.

## 3. End-to-end flow of a single review

The diagram below traces one review from trigger to HITL surfacing, with audit-trail writes annotated at each stage.

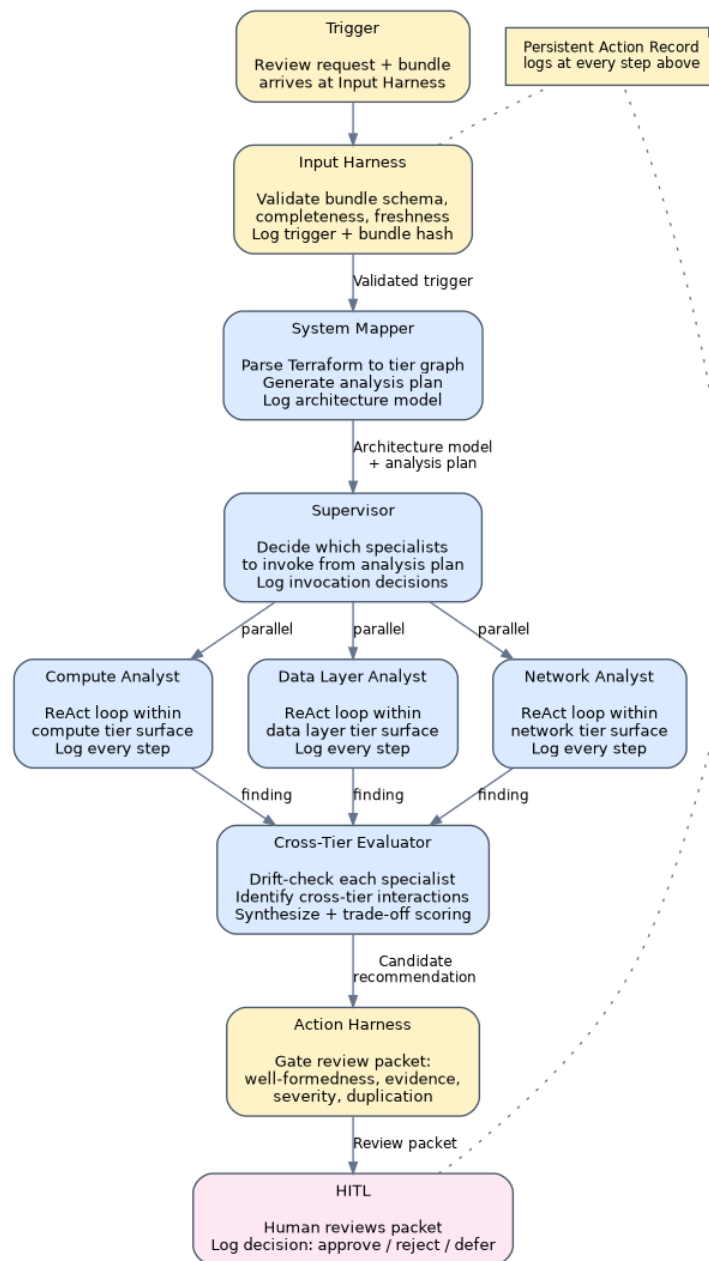


Figure 3: End-to-end flow of a single review cycle

The Persistent Action Record participates in every stage. By the end of a review, the audit trail contains a complete, ordered, append-only record of everything that happened — enough to deterministically replay the entire reasoning chain.

## 4. Tier specialist internals — the ReAct loop

Each Tier Specialist runs a ReAct loop against a constrained read surface (the surface is defined formally in docs/04-react-tool-surface.md, which references the dataset's §2.6 read contract).

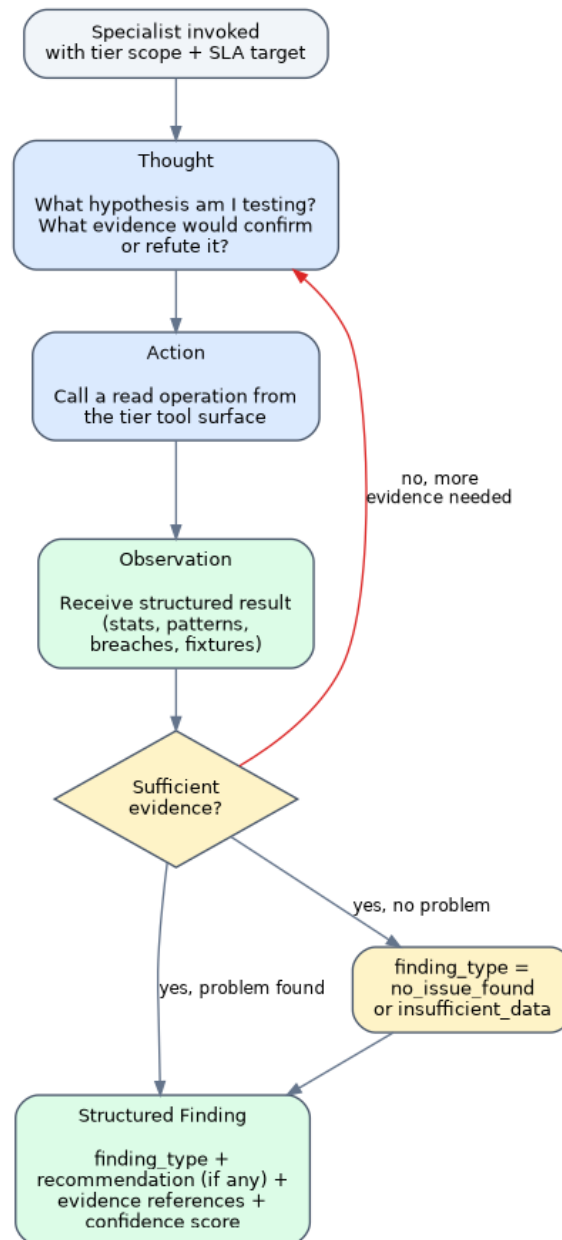


Figure 4: Tier specialist ReAct loop

The ReAct trace itself is what makes the audit trail meaningful. A specialist that produces a recommendation from a single LLM call leaves nothing to audit. A specialist that hypothesizes, queries, observes, refines, and concludes produces a reasoning trace that a human reviewer can actually engage with. See



`docs/09-design-decisions.md` for the full justification.

The `finding_type` enum has three values — `issue_found`, `no_issue_found`, `insufficient_data` — explicitly so that specialists are not biased toward inventing problems. See `docs/02-agents.md`.

## 5. Cross-Tier Evaluator internals — drift-check, then synthesize

The Evaluator's role is the architectural keystone. Its three sub-steps are sequenced deliberately.

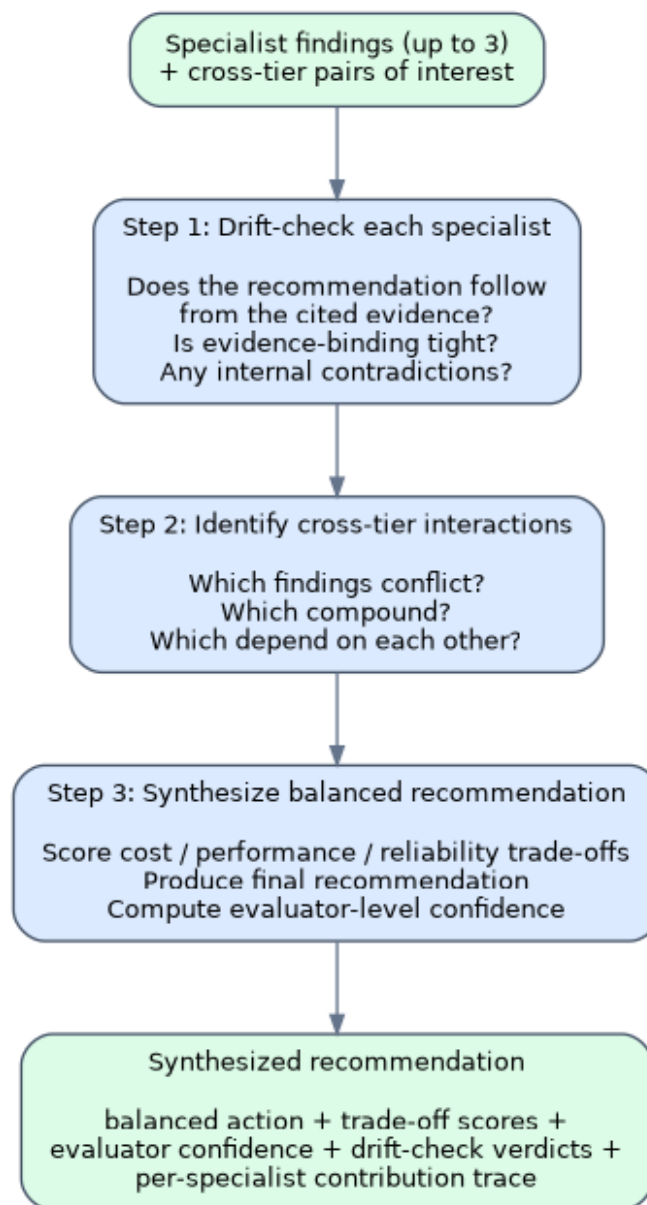


Figure 5: Cross-Tier Evaluator internals

Step 1 is what makes the Evaluator a quality gate, not just a synthesis layer. A specialist that hallucinates a recommendation that doesn't follow from its own evidence is caught here. A specialist that contradicts what cross-tier evidence implies is caught in Step 2.

Correlated multi-specialist drift — when all three specialists drift in the same direction — is **not** caught by the Evaluator. That class of failure is structurally caught by HITL review, with the audit trail providing the basis for human adjudication. See [docs/05-confidence-and-drift.md](#).

## 6. Where each harness shows up in the flow

Mapping the four harnesses to the end-to-end flow:

| Stage of flow              | Input Harness                                       | Reasoning Harness                                                                                 | Action Harness                                                                                       | Persistent Action Record                               |
|----------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Trigger + bundle ingest    | <b>Active</b> — validates bundle, schema, freshness | —                                                                                                 | —                                                                                                    | Logs trigger + bundle hash                             |
| System Mapper              | Validates Terraform parses                          | Enforces architecture-model schema                                                                | —                                                                                                    | Logs architecture model + analysis plan                |
| Supervisor decisions       | —                                                   | —                                                                                                 | —                                                                                                    | Logs specialist invocation decisions                   |
| Tier Specialist ReAct loop | —                                                   | <b>Active</b> — structured reasoning, evidence-binding, specialist-level confidence               | <b>Active</b> — gates which read operations the specialist can call (tier-scoped)                    | Logs every tool call, every reasoning step             |
| Cross-Tier Evaluator       | —                                                   | <b>Active</b> — structured drift-check + synthesis, evaluator-level confidence, trade-off scoring | —                                                                                                    | Logs drift-check verdicts, trade-off scores, synthesis |
| Recommendation surfacing   | —                                                   | —                                                                                                 | <b>Active</b> — gates recommendation well-formedness, evidence completeness, severity classification | Logs gate verdict                                      |
| HITL decision              | —                                                   | —                                                                                                 | —                                                                                                    | Logs HITL decision                                     |

The Reasoning Harness does the most work. The Action Harness is intentionally narrow. The Persistent Action Record participates everywhere. The Input Harness is concentrated at the front door.

## 7. What this architecture is *not*

The diagrams are not a microservice deployment diagram. The system can be deployed as a single Python process for the portfolio implementation; the architectural separation is logical, not infrastructural.

The diagrams are not a state-machine specification. LangGraph or an equivalent framework will implement the actual control flow; the diagrams describe the conceptual structure that the implementation realizes.

The Supervisor is not a router. It makes real workflow decisions — which specialists to invoke, when to retry on low confidence, when to escalate ambiguous results — described in [docs/02-agents.md](#).

The Cross-Tier Evaluator is not a winner-picker. When specialist findings conflict, the Evaluator surfaces the conflict and reasons through the trade-off, rather than silently selecting one. See [docs/05-confidence-and-drift.md](#).

## 2. Design Principles

---

This document captures the principles that shaped every other decision in this project. The principles are not preferences — they are commitments that, taken together, define what kind of system this is and what it deliberately is not.

---

### 1.1 Recommender, not actuator

The system's identity is **recommendation under human-in-the-loop review**. Every recommendation flows to a human. The system itself never changes infrastructure state.

This is a deliberate choice, not a limitation. Two reasons:

**Identity clarity.** A system that recommends *and* acts is two systems wearing one face. The reasoning required to produce a sound recommendation is different from the safety machinery required to execute one. Conflating them dilutes both. Cloud governance is specifically a domain where human judgment on recommendations is the operating model — FinOps engineers, SREs, and platform teams want recommendations they can vet, not automated changes they have to audit after the fact.

**Architectural honesty.** Because recommendations always pass through a human, the Action Harness has a small and well-defined job: gate the *quality* of the recommendation surfaced for review, and gate which read operations agents can call during analysis. The Action Harness is not the place where catastrophic decisions are prevented — that's HITL's job. Resisting the temptation to make the Action Harness bigger is part of the architectural signal. See [03-harnesses.md](#).

A consequence of this principle is that the audit trail's purpose is not to support rollback or incident response. Its purpose is to make every recommendation **replayable** so that humans, weeks or months after the fact, can reconstruct exactly why a recommendation was produced. See [06-persistent-action-record.md](#).

---

### 1.2 Hierarchical multi-agent — earned, not decorative

A single LLM with all the telemetry, sidecar metadata, and Terraform in one prompt could produce a recommendation. The question is whether that recommendation would be defensible.

The system uses four agents because each one has a job no other agent can do well:

- **System Mapper** translates infrastructure description into an analysis plan — a task that benefits from deterministic parsing rather than LLM reasoning.
- **Tier Specialists** perform deep, scoped analysis with reasoning constrained to a single domain. Compute, data layer, and network optimization are genuinely different disciplines with different telemetry, different optimization levers, and different failure modes. Forcing one agent to be expert in all three produces shallower reasoning across the board.
- **Supervisor** decides which specialists to invoke based on the application's actual topology, and orchestrates retries and escalations.
- **Cross-Tier Evaluator** synthesizes across specialists *and* checks each one for drift — a job that requires a wider view than any individual specialist has, and that benefits from a stronger model.

The architectural test is: would removing this agent leave the system meaningfully worse? For all four, yes. See `02-agents.md` for the per-agent justification.

This is a stronger justification for multi-agent than most demos can offer. Many "multi-agent" systems are single agents with role-switching prompts. This one has genuinely separate concerns.

## 1.3 Failure modes are about quality, consistency, and accountability — not adversarial threats

A common mistake in agent system design is importing an adversarial threat model from a different domain. This system has no external user input — the inputs are Terraform files and telemetry from internal infrastructure. Prompt injection is not a meaningful threat. Scope creep is architecturally prevented because each specialist is bounded by its tier's input data and read surface.

The failure modes that actually apply to this system:

| Failure mode                                          | What it looks like                                                                                                                                                | Where it is addressed                                                                                                                                                                                                              |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Garbage-in recommendations</b>                     | System Mapper misreads Terraform; telemetry is incomplete or stale; sidecar fields are missing. Every downstream specialist analyzes a wrong model of the system. | Input Harness validates bundle and Terraform at ingest.                                                                                                                                                                            |
| <b>Specialist hallucination within a tier</b>         | A specialist recommends an action without sufficient evidence, or cites evidence it did not actually observe.                                                     | Reasoning Harness enforces structured output with evidence-binding and a minimum-evidence threshold. ReAct trace makes the hallucination visible in the audit trail. Cross-Tier Evaluator drift-checks the specialist's reasoning. |
| <b>Cross-tier inconsistency / trade-off blindness</b> | Specialists individually produce locally-correct recommendations that conflict when combined.                                                                     | Cross-Tier Evaluator explicitly identifies cross-tier interactions and scores trade-offs across cost/performance/reliability.                                                                                                      |
| <b>Recommendation surfacing quality</b>               | A recommendation is technically valid but mis-classified for severity, duplicates a prior recommendation, or is missing context the human reviewer needs.         | Action Harness gates well-formedness, evidence completeness, and severity classification of the review packet.                                                                                                                     |
| <b>Lack of decision provenance</b>                    | Three months after a recommendation was acted on, nobody can reconstruct why the system recommended it.                                                           | Persistent Action Record logs every decision, every reasoning step, every evidence citation. Replayable end-to-end.                                                                                                                |

Two failure modes are deliberately **not** mitigated by the harnesses, because the architecture handles them differently:

- **Scope creep across tiers** is not a runtime concern because each specialist's read surface is structurally limited to its own tier. The architecture prevents the failure rather than catching it.
- **Correlated multi-specialist drift** — where all three specialists drift in the same direction and the Evaluator therefore has no internal conflict to flag — is caught by HITL review. The audit trail makes the drift visible to humans even when the Evaluator misses it.

Naming what is mitigated and what is not is part of the architectural signal. Pretending the harnesses defend against everything is the kind of overclaim a senior reviewer notices immediately.

---

## 1.4 Synthetic data, deliberately

The system is evaluated against an 18-scenario synthetic dataset specified in the Cloud Governance Dataset Generation Plan v3. Three reasons:

**Ground truth.** Every scenario has a known target recommendation. Real cloud telemetry would not.

**Coverage of trade-off cases.** Real telemetry does not deterministically exercise the cost-vs-reliability, mixed-state, restraint, and diagnostic-deferral cases that distinguish a sophisticated recommender from a simple one. The synthetic dataset is hand-crafted to exercise exactly these cases.

**Decoupling from cloud account state.** A portfolio implementation should run locally on any machine without an AWS account, billing setup, or cluster provisioning. The synthetic dataset makes this true.

The dataset is reverse-engineered from target recommendations. That is acknowledged as a deliberate evaluation device, not a limitation — see [07-evaluation-methodology.md](#).

---

## 1.5 The harnesses provide properties, not defenses

The four harnesses — Input, Reasoning, Action, Persistent Action Record — are described throughout this repository as providing **structure, safety, and observability**. The vocabulary matters.

The harnesses are not five defenses mapped to five threats. They are four architectural components that, together, make a hierarchical multi-agent recommender trustworthy enough for governance work.

- **Structure** comes from explicit schemas, evidence-binding, and ReAct discipline.
- **Safety** comes from quality-gating of recommendations and constraint of tool surfaces — not from adversarial threat defense.
- **Observability** comes from the append-only audit trail and the replayability guarantee.

Framing the harnesses this way is what allows the architecture to be honest about what it does and does not do. See [03-harnesses.md](#).

---

## 1.6 Specialization over scale

The system uses two LLM tiers: a fast, cheap model (Claude Haiku) for the Tier Specialists, and a stronger model (Claude Sonnet) for the Cross-Tier Evaluator. The choice is deliberate — specialists do many ReAct steps where speed and cost matter, while the Evaluator does the hardest reasoning once per review where stronger capability earns its cost.

A single-model architecture would be simpler but would either over-spend on specialists or under-power the Evaluator. The two-tier split is itself a senior signal — it demonstrates cost-aware design. See [08-llm-and-cost.md](#).

---

## 1.7 The trade-offs are part of the deliverable

A common failure mode of portfolio projects is presenting an architecture as if it were inevitable. This project's architecture has alternatives at almost every decision point — zero-shot specialists instead of ReAct, vector storage instead of relational for the audit trail, a single-tier LLM mix, a sidecar-only System Mapper, an inflated Action Harness.

Each of those alternatives was considered and rejected for specific reasons. Those reasons live in [09-design-decisions.md](#). The ability to articulate why a chosen design was preferred over its alternatives is, in interview terms, more valuable than the chosen design itself.

## 3. Agents

---

Four agents, each with a single clear job. This document defines what each agent does, what it does *not* do, and why it earns its place in the architecture.

The agent topology and the end-to-end flow live in `../ARCHITECTURE.md`. This document focuses on roles, responsibilities, inputs, outputs, and the rationale that justifies each agent as a separate layer.

---

### 2.1 Supervisor

#### Role

The Supervisor orchestrates the governance review workflow. It is the entry point of every review and the layer that decides how the rest of the system is invoked.

#### Responsibilities

- **Trigger handling.** Accepts review requests (scheduled, change-driven, or on-demand). Validates that the bundle is well-formed before any downstream agent runs.
- **Architecture model retrieval.** Invokes the System Mapper to obtain the tier dependency graph and analysis plan for the application.
- **Specialist invocation decisions.** Based on the analysis plan, decides which tier specialists to invoke for this application. Skips specialists for tiers not present in the topology.
- **Low-confidence handling.** When a specialist returns a finding with confidence below the Reasoning Harness threshold, the Supervisor decides whether to retry (with additional context or a different decomposition), pass the finding through with a confidence flag, or defer to human review.
- **Aggregation.** Collects specialist findings and forwards them to the Cross-Tier Evaluator along with the cross-tier pairs of interest from the analysis plan.

#### Inputs

- Review trigger metadata (timestamp, application ID, trigger type).
- Validated bundle from the Input Harness.

#### Outputs

- Specialist invocation commands (which specialists, with what scope).
- Aggregated findings package for the Cross-Tier Evaluator.

#### Why this is a separate agent

A pass-through router does not earn its own agent layer. This Supervisor earns its layer because it makes real workflow decisions: which subset of the three specialists is appropriate for this application's topology, how to handle low-confidence findings, when to escalate. These are decisions that LangGraph state-machine routing alone could not make without an LLM-driven decision layer above it.



The Supervisor's continual-monitoring story — deciding when applications are due for review based on time, recent changes, or anomalies — is described as design intent but is **scoped to scenario-batch invocation for the portfolio implementation**. The full trigger logic of a production system is out of scope. This is called out honestly in `10-limitations.md`.

### What the Supervisor does *not* do

- It does not analyze telemetry. That is the specialists' job.
- It does not synthesize recommendations. That is the Evaluator's job.
- It does not gate recommendations against the human reviewer. That is the Action Harness's job.

## 2.2 System Mapper

### Role

The System Mapper translates an infrastructure description (Terraform) into an analysis plan that the rest of the system reasons against.

### Responsibilities

- **Terraform parsing.** Parses Terraform definitions to extract resources, their attributes, and the dependency relationships between them.
- **Tier classification.** Maps parsed resources to the system's three tier categories — compute, data layer, network (cache when present).
- **Dependency graph construction.** Identifies which tiers communicate with which, and characterizes the connection type (synchronous query, read-through cache, asynchronous queue, etc.).
- **Analysis plan generation.** Produces a structured plan specifying which specialists are required and which cross-tier pairs the Evaluator should pay attention to.

### Inputs

- Terraform definition files for the application under review.
- Sidecar metadata (for cross-referencing instance classes and counts).

### Outputs

A structured **architecture model** containing:

- The list of tiers present in this application, with their tier type and key configuration attributes.
- The list of dependencies between tiers, characterized by type.
- An analysis plan specifying which specialists to invoke and which cross-tier pairs are of interest for synthesis.

### Why this is a separate agent

The sidecar metadata already contains instance classes, counts, replica counts, and pool sizes. A System Mapper that only read the sidecar would be a JSON deserializer. The System Mapper earns its own layer

because it does work the sidecar cannot do: it parses *how the tiers connect*, which is encoded in Terraform's `depends_on` and reference relationships and is not present in the sidecar's flat configuration data.

The output — the analysis plan — is what makes the Supervisor's decisions meaningful. Without the analysis plan, the Supervisor would have to invoke all three specialists for every review regardless of topology. With it, the Supervisor invokes only the specialists relevant to this application.

For the portfolio implementation, Terraform stubs are hand-authored alongside each scenario to match the sidecar's stated configuration. The System Mapper's parsing logic is realistic and uses a standard Terraform parsing library (e.g. `python-hcl2`), but the inputs are crafted to be unambiguous.

### What the System Mapper does *not* do

- It does not validate Terraform syntax or correctness. That is not the system's job.
- It does not recommend any architectural changes. It describes; it does not prescribe.
- It does not reason over telemetry. It only reads the Terraform definition and the sidecar.

## 2.3 Tier Specialists (×3)

Three specialists run in parallel, each scoped to one tier: **Compute Analyst**, **Data Layer Analyst**, **Network & Integration Analyst**. They share an architectural pattern and differ only in the read surface they operate against (the per-tier surface is defined in `04-react-tool-surface.md`).

### Role

Each specialist performs deep, tier-bounded analysis of the application's behavior in its tier, using ReAct reasoning against a constrained read surface, and produces a structured finding.

### Responsibilities

- **ReAct loop.** Form a hypothesis about the tier's behavior, query the read surface for evidence, observe results, refine, and conclude. Each cycle is logged to the audit trail.
- **Evidence-bound reasoning.** Every conclusion must cite the specific read operations and their results that justified it. The Reasoning Harness enforces this structurally — see `03-harnesses.md`.
- **Confidence scoring.** Produce a specialist-level confidence score combining evidence sufficiency and within-tier pattern strength. See `05-confidence-and-drift.md`.
- **Structured finding output.** Emit a finding with an explicit `finding_type` — one of `issue_found`, `no_issue_found`, `insufficient_data` — plus the recommendation (if any), evidence references, and confidence score.

### Inputs

- Tier scope (which tier this specialist owns).
- Application context (SLA target, business context, current configuration for the tier).
- Access to the tier's read surface.

### Outputs

A structured specialist finding — see `03-harnesses.md` for the schema.

## Why three specialists, not one

Cloud optimization in 2026 is genuinely three different disciplines:

- **Compute** reasoning works against CPU/memory utilization patterns, scaling policy effectiveness, load balancer fairness, and application-level latency. Recommendations are about instance sizing, scaling policies, and load distribution.
- **Data layer** reasoning works against query performance, connection management, replication topology, and cache coherence. Recommendations are about instance sizing, query optimization, replica strategy, and pool tuning.
- **Network** reasoning works against inter-service latency, error rates, and topology choices. Recommendations are about peering, private links, retry strategies, and routing.

The optimization vocabularies, the failure signatures, and the appropriate interventions are different. A single agent forced to reason across all three would either over-generalize or skip depth in two of them.

Running them in parallel is a real architectural choice: it cuts wall-clock time, and it structurally prevents one specialist's reasoning from influencing another's. Specialist independence is what makes the Cross-Tier Evaluator's drift-check meaningful — three specialists that have not seen each other's reasoning cannot have correlated within the analysis itself.

## The three-valued finding\_type

A common failure mode of agent systems is producing a recommendation when the right answer is "no action." Specialists in this system are explicitly designed to handle three outcomes:

- `issue_found` — sufficient evidence supports a specific recommendation.
- `no_issue_found` — the tier is operating within expected ranges and no action is warranted. This is a valid, full-confidence finding.
- `insufficient_data` — the available evidence is not sufficient to make a determination, and a recommendation would be a leap.

The Reasoning Harness enforces an evidence threshold before any `issue_found` finding can be emitted. Without sufficient evidence, the specialist must emit `no_issue_found` or `insufficient_data`. This is what gives the system its restraint capability — see the scenarios in the dataset doc tagged with the "Restraint" evaluation property.

## What specialists do *not* do

- They do not access other tiers' data. The tool surface is structurally scoped.
- They do not synthesize across tiers. That is the Evaluator's job.
- They do not gate their own output for surfacing. That is the Action Harness's job.

---

## 2.4 Cross-Tier Evaluator

### Role

The Cross-Tier Evaluator is the synthesis layer and the system's primary quality gate against single-specialist drift. It is the architectural keystone: without it, three specialists produce three findings with no mechanism to reconcile them.

## Responsibilities

The Evaluator runs three sub-steps in sequence:

**Step 1 — Drift-check each specialist.** For each specialist finding, the Evaluator examines:

- Does the recommendation follow from the cited evidence?
- Is the evidence-binding tight, or does the recommendation reach beyond what the evidence supports?
- Are there internal contradictions within the specialist's reasoning trace?

Drift-check verdicts are logged to the audit trail per specialist. A specialist whose finding fails drift-check has its weight in synthesis reduced, and the failure is surfaced in the review packet so the human reviewer sees it.

**Step 2 — Identify cross-tier interactions.** Using the cross-tier pairs of interest from the System Mapper's analysis plan, the Evaluator examines:

- Which specialist findings conflict directly (e.g. Compute recommends downsize while Data Layer evidence shows the database is the bottleneck)?
- Which findings compound (e.g. cache miss cascade implicates both cache and database tiers, requiring a coordinated recommendation)?
- Which findings depend on each other (e.g. fixing the data layer first changes what the compute recommendation should be)?

**Step 3 — Synthesize a balanced recommendation.** With drift-checked specialist findings and identified cross-tier interactions, the Evaluator produces:

- A final recommendation that balances cost, performance, and reliability dimensions explicitly.
- Trade-off scores across the three dimensions (each scored on its own scale, with the trade-off articulated rather than collapsed into a single number).
- An evaluator-level confidence score that combines specialist confidences with cross-tier consistency and trade-off resolution clarity. See [05-confidence-and-drift.md](#).
- A reasoning trace that cites which specialist evidence supported the synthesis.

## Inputs

- Aggregated specialist findings (1 to 3 of them, depending on which specialists the Supervisor invoked).
- Cross-tier pairs of interest from the System Mapper's analysis plan.
- Application context (SLA target, business context).

## Outputs

A synthesized recommendation containing the balanced action, trade-off scores, evaluator confidence, drift-check verdicts per specialist, and per-specialist contribution traces. This is the input to the Action Harness's recommendation gate.

## LLM-as-judge — judge of what

The Evaluator uses LLM-as-judge in two senses:

- **Judge of specialist reasoning quality** in Step 1 — assessing whether each specialist's recommendation follows from its cited evidence.
- **Judge of trade-offs** in Step 3 — weighing cost vs. performance vs. reliability when they point in different directions.

Both are real judging jobs. This is what makes the Evaluator the right place to use a stronger LLM (Claude Sonnet) than the specialists do.

Diagnostic deferral

For scenarios where capacity metrics are normal but latency is elevated across tiers simultaneously (the architectural shape of Scenario 17 in the dataset doc), the Evaluator can produce a `recommend_diagnosis` output rather than a balanced action. This is the system's diagnostic-deferral capability — recognizing when the right next step is a distributed trace analysis to identify root cause, not a scaling decision.

The Reasoning Harness output schema for the Evaluator includes this as a first-class output mode alongside the standard `recommend_action` and `recommend_no_action` modes.

What the Evaluator does *not* do

- It does not catch correlated multi-specialist drift. If all three specialists drift in the same direction with no internal conflict, the Evaluator has no signal to flag. HITL review catches this class of failure, supported by the audit trail.
- It does not act on its own recommendation. The recommendation passes through the Action Harness gate and into HITL.
- It does not override the System Mapper's analysis plan. Cross-tier pairs of interest are taken as input, not chosen by the Evaluator.

2.5 Why this specific agent set

The architecture was settled by asking, for each candidate agent, whether removing it would leave the system meaningfully worse:

| Agent                | What removing it would cost                                                                                                                                                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Supervisor           | Specialists would always run for every tier regardless of topology; low-confidence handling would have nowhere to live; the trigger-handling story would be invisible.                                                                                     |
| System Mapper        | The Supervisor would have nothing to base specialist-invocation decisions on; cross-tier pairs of interest would have to be invented or hardcoded; the "Terraform → analysis plan" translation step would disappear and with it the architectural realism. |
| Tier Specialists (3) | A single specialist trying to cover three disciplines would either over-generalize or skip depth in two. Within-tier audit trail richness would collapse.                                                                                                  |

| Agent                | What removing it would cost                                                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cross-Tier Evaluator | The system would have three independent findings with no synthesis layer, no drift-check, no trade-off scoring, and no balanced final recommendation. |

Each agent earns its place. None is decorative.

## 4. Harnesses

The four harnesses provide structure, safety, and observability across the agent system. They are not adversarial threat defenses — see `01-design-principles.md` §1.3 and §1.5 for why this framing matters.

This document describes what each harness provides, where it applies in the system, and why it is shaped the way it is.

### 3.1 The four harnesses at a glance

| Harness                  | What it provides                                                                                | Where it applies                                                                    |
|--------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Input Harness            | First-line validation of bundle data and review triggers                                        | At bundle ingest, before any agent reasons                                          |
| Reasoning Harness        | Structured reasoning output, evidence-binding, two-level confidence scoring, trade-off analysis | At every specialist ReAct step and every Evaluator turn                             |
| Action Harness           | Tool-surface control and recommendation quality gating                                          | Around every tool call during analysis, and at the final recommendation before HITL |
| Persistent Action Record | Append-only audit trail with replayable reasoning chains                                        | Throughout — every stage writes                                                     |

The harnesses are layered, not nested. A single agent invocation may interact with all four: the Input Harness validated its inputs, the Reasoning Harness shapes its reasoning and output, the Action Harness gates its tool calls, and the Persistent Action Record logs every step.

### 3.2 Input Harness

#### What it provides

Validation that the bundle (telemetry + sidecar) and the review trigger are well-formed and complete enough to support downstream analysis. Garbage in is stopped here so no specialist wastes a reasoning cycle on bad data.

#### What it validates

- **Schema conformance.** Telemetry tier arrays conform to the schemas defined in the dataset doc §2.2. Sidecar conforms to §2.3.
- **Record completeness.** Each non-empty tier array has the expected 1,344 records.
- **Timestamp continuity.** Timestamps are monotonic, 15 minutes apart, no gaps or duplicates.
- **Cross-tier alignment.** Timestamps align index-by-index across the tier arrays used in this bundle.

- **Sidecar field presence.** Sidecar fields referenced by the read contract (`top_queries` for scenarios that need them, `per_instance_imbalance` for the load-balancer scenario, etc.) are present and non-empty.
- **Trigger legitimacy.** The review trigger is well-formed and identifies an application that is in scope.
- **Terraform parseability.** The application's Terraform definition parses without error. (Validation of *correctness* is out of scope — see `02-agents.md` §2.2.)

## Where it applies

At the front door. Every review request passes through the Input Harness before the Supervisor invokes any other agent. Failures here produce a clear rejection with the specific validation that failed, logged to the audit trail. Downstream agents never see a malformed bundle.

## Why this is its own harness

A harness for input validation may look unsophisticated compared to reasoning or audit. It is included as a distinct harness because the alternative — letting downstream agents discover bad inputs through their own failures — produces useless audit trails ("specialist could not find evidence" when the real failure was that telemetry was incomplete). Validating at the front door means downstream audit entries are about real reasoning, not about data quality artifacts.

## What it does *not* do

- It does not validate semantic correctness of inputs. A bundle that passes schema and completeness checks but contains unrealistic values will still pass — semantic plausibility is the dataset generation pipeline's job, not the consumer's.
- It does not perform adversarial input filtering. There is no external user input in this system; the inputs are infrastructure description and synthesized telemetry.

# 3.3 Reasoning Harness

## What it provides

The cognitive infrastructure that makes specialist and Evaluator reasoning credible. The Reasoning Harness is the heaviest of the four harnesses — most of the system's intelligence lives in the structures it enforces.

## What it enforces

**Structured output with evidence-binding.** Every specialist finding and every Evaluator synthesis is emitted as a structured object with a fixed schema. Critically, every recommendation field within the structure must reference the specific read operations (and their results) that justified it. A recommendation without evidence references is structurally rejected before it can be emitted.

The output schema for a specialist finding:

| Field                     | Purpose                                                                                                                                                                               |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>finding_type</code> | One of <code>issue_found</code> , <code>no_issue_found</code> , <code>insufficient_data</code> . Explicitly three-valued to prevent action bias — see <code>02-agents.md</code> §2.3. |



| Field                              | Purpose                                                                                                                      |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>recommendation</code>        | The specific action recommended (only populated when <code>finding_type = issue_found</code> ).                              |
| <code>evidence_refs</code>         | List of read-operation results that justified the recommendation. Required to be non-empty when a recommendation is present. |
| <code>reasoning_trace</code>       | The ReAct hypothesize/act/observe trace that led to the finding.                                                             |
| <code>specialist_confidence</code> | A scalar combining evidence sufficiency and within-tier pattern strength.                                                    |
| <code>confidence_breakdown</code>  | The sub-signals contributing to the confidence score, named individually.                                                    |

The Evaluator's output schema extends this with cross-tier interactions, trade-off scores, drift-check verdicts per specialist, and evaluator-level confidence.

**Evidence sufficiency threshold.** A `recommendation` field can only be populated if `evidence_refs` exceeds a minimum count *and* meets the specialist's evidence threshold for its domain. Falling below either threshold forces the specialist to emit `no_issue_found` or `insufficient_data`.

**Two-level confidence scoring.** Specialists produce a specialist-level confidence; the Evaluator produces an evaluator-level confidence. The two levels measure different things and combine differently. See `05-confidence-and-drift.md`.

**Trade-off analysis.** The Evaluator produces explicit trade-off scores across cost, performance, and reliability dimensions. Each is scored on its own scale, rather than collapsed into a single composite. The trade-off is articulated in the reasoning trace so the human reviewer can see how the dimensions were weighed.

## Where it applies

At every specialist ReAct turn (each `thought` → `action` → `observation` cycle is subject to the harness's structuring) and at every Evaluator step (drift-check, cross-tier identification, synthesis). The harness does not apply to the Supervisor or System Mapper, whose outputs follow simpler workflow-orchestration and parsing schemas respectively.

## Why this is the heaviest harness

The Reasoning Harness is where the architectural claim "this system reasons soundly" becomes a structural property rather than a hope. Without evidence-binding, recommendations are leaps. Without the three-valued `finding_type`, specialists invent problems. Without two-level confidence, the drift-check has nothing to gate on. Without trade-off scoring, the synthesis is "pick a winner" rather than "balance dimensions."

Most of what makes this project interview-worthy lives in this harness. The specialists and the Evaluator are vehicles for the reasoning the harness enforces.

## What it does *not* do

- It does not generate reasoning. Agents do that. The harness enforces shape and constraint.

- It does not score recommendations against the target. That is the Evaluation Methodology — see [07-evaluation-methodology.md](#).
- It does not audit reasoning. That is the Persistent Action Record's job.

## 3.4 Action Harness

### What it provides

Two scoped jobs:

- 1 **Tool-surface control during analysis.** Each agent can only invoke the read operations appropriate to its scope. Tier specialists are limited to their own tier's read surface; cross-tier operations are reserved for the Evaluator. The System Mapper has its own tool surface for Terraform parsing.
- 1 **Recommendation quality gate before HITL.** The final synthesized recommendation, before being surfaced to the human reviewer, passes through a gate that checks:
  - Well-formedness of the review packet (all required fields populated, evidence chain intact).
  - Evidence completeness (the recommendation's evidence references actually resolve to logged read operations in the audit trail).
  - Severity classification (recommendations are classified low/medium/high for HITL routing — higher severity gets more prominent placement and stricter human-review expectations).
  - Duplication check (a recommendation that exactly matches a recent prior recommendation gets flagged so the human sees the repetition).

### Why intentionally narrow

The Action Harness in a system that actuates state changes (rather than recommends) would be much larger — whitelisting executable actions, parameter validation against external APIs, idempotency keys, rollback paths. None of that applies here. The system's identity is recommendation under HITL; there is no actuation to gate.

Inflating the Action Harness to look bigger would dilute the system's identity. The discipline of keeping it small is part of the architectural signal. See [01-design-principles.md §1.1](#) and [09-design-decisions.md](#).

### Where it applies

- Around every tool call by every agent (tool-surface control).
- At the final recommendation, between the Evaluator's synthesis and the HITL surfacing (recommendation quality gate).

### What it does *not* do

- It does not actuate state changes. The system is a recommender — see [01-design-principles.md §1.1](#).
- It does not gate the Evaluator's synthesis content. The Reasoning Harness shapes that. The Action Harness only checks the resulting review packet for completeness and classification.
- It does not enforce role-based access control. Role-based filtering is out of scope for a system that produces recommendations for a single human reviewer per review. (Original Project 2 had role-based filtering because it was an IT support agent with multiple user personas; this project does not.)

## 3.5 Persistent Action Record

### What it provides

An append-only audit trail across every agent and every decision, supporting deterministic replay of any recommendation back to its evidence chain.

### What gets logged

Every significant event in a review cycle becomes a record. The schema captures, at minimum:

- **Trigger events** — review request, bundle hash, validation outcomes from the Input Harness.
- **System Mapper events** — architecture model produced, analysis plan generated.
- **Supervisor decisions** — which specialists invoked, why; low-confidence handling decisions; retries.
- **Specialist ReAct steps** — every thought, action, observation. Each tool call logged with its parameters and result.
- **Specialist findings** — finding\_type, recommendation (if any), evidence\_refs, reasoning\_trace, confidence breakdown.
- **Evaluator events** — drift-check verdicts per specialist, cross-tier interactions identified, trade-off scores, synthesis output, evaluator confidence breakdown.
- **Action Harness events** — recommendation gate verdict, severity classification, duplication check result.
- **HITL events** — review packet surfaced, human decision (approve/reject/defer) with timestamp and any reviewer notes.

Every record carries a stable identifier, the agent or harness that emitted it, and foreign-key references to upstream records in the chain. The full chain from trigger to HITL decision is reconstructable from these references.

### Append-only by design

Records are never updated or deleted. Corrections to a recommendation produce new records that reference the original. This is what makes "replayable" a real property rather than a hope: the historical state of any past review is the records that exist at that point in the log, and they cannot be retroactively rewritten.

### Source traceability is first-class

Every recommendation in the review packet carries explicit references to the audit-trail records that justified it. An auditor — or future-you — can trace any recommendation back to the specific specialist reasoning steps, the specific tool-call results, and the specific cross-tier evidence that produced it. This is the "replayability" claim made concrete.

### Storage choice

SQLite for the portfolio implementation, with the schema designed so Postgres is a drop-in upgrade for a production deployment. SQLite means the audit trail demo runs anywhere with no infrastructure setup — a real advantage for a portfolio project. The senior signal is the schema discipline, not the database engine choice.

A vector database was considered and rejected. The audit trail's access patterns are relational (foreign-key chains, structured queries, append-only semantics), not similarity-based. See [09-design-decisions.md](#).

Full schema design lives in [06-persistent-action-record.md](#).

## Why this is its own harness, not bolted-onto-agent logging

This is the original Project 2 framing and it carries forward unchanged for this project: treating the audit trail as its own architectural component, rather than as logging-bolted-onto-the-agent, reflects how production systems are actually built. The audit trail is the artifact a compliance or governance reviewer engages with. It deserves its own schema, its own write discipline, and its own correctness story.

It is also the artifact that turns "I built an agent system" into "I built an agent system that could pass a governance review." The other three harnesses depend on this one for accountability — they enforce structure and quality, but the audit trail is what makes the enforcement *verifiable after the fact*.

## What it does *not* do

- It is not an event bus. Agents do not communicate through the audit trail; they communicate through the Supervisor's orchestration. The audit trail is a parallel write-only stream.
- It is not a knowledge base. The agents do not query the audit trail for past scenarios as input to current reasoning. (That would be a different kind of system — a system with memory, not just observability.)
- It is not a vector store. See [09-design-decisions.md](#).

---

## 3.6 How the harnesses interact

The harnesses are independent in scope but cooperative in practice. A representative interaction:

A Compute Analyst invokes a tool call to detect threshold breaches on `cpu_p95`. The **Action Harness** validates that this read operation is in the Compute Analyst's scope (it is). The tool call runs and returns results. The **Reasoning Harness** requires that any conclusion citing this tool call include the call as an `evidence_ref`. The **Persistent Action Record** logs the tool call, the parameters, the result, and (when the specialist concludes) the finding that cites it.

Four harnesses, one tool call, four properties enforced: scope (Action Harness), evidence-binding (Reasoning Harness), audit (Persistent Action Record), and — implicitly — the trust that the tool call's inputs were valid (Input Harness validated them at bundle ingest).

This is what "structure, safety, and observability" means in practice. Not five defenses against five threats, but four properties enforced uniformly across the system.

# 5. ReAct Tool Surface

The Tier Specialists and the Cross-Tier Evaluator reason against a constrained read surface defined in the Cloud Governance Dataset Generation Plan v3 §2.6. This document describes how that read contract is partitioned by agent role, and how the surface shapes the audit trail.

The read operations themselves are defined in the dataset doc. This document is about *who can call what, when, and why*.

## 4.1 The read contract, as agent tool surface

The dataset doc §2.6 specifies fifteen abstract read operations across three groups:

- **Telemetry reads** (5 operations) — time-series, summary statistics, time pattern, threshold detection, distribution.
- **Sidecar reads** (8 operations) — business context, SLA target, configuration, monthly cost, top queries, top cache keys, per-instance breakout, before/after evidence.
- **Cross-tier reads** (2 operations) — correlation evidence, pattern alignment.

The Action Harness gates which operations each agent can invoke. The partitioning below is not a suggestion — it is enforced.

## 4.2 Per-agent tool surfaces

### Compute Analyst

| Available operations      | Tier scope |
|---------------------------|------------|
| get_time_series           | compute    |
| get_summary_statistics    | compute    |
| get_time_pattern          | compute    |
| detect_threshold_breaches | compute    |
| get_metric_distribution   | compute    |
| get_business_context      | (no tier)  |
| get_sla_target            | (no tier)  |
| get_configuration         | compute    |
| get_monthly_cost          | (no tier)  |
| get_before_after_evidence | (no tier)  |

| Available operations                   | Tier scope                                                                              |
|----------------------------------------|-----------------------------------------------------------------------------------------|
| <code>get_per_instance_breakout</code> | (no tier — but only meaningful for compute scenarios; returns null for other scenarios) |

The Compute Analyst cannot invoke read operations against `database`, `network`, or `cache` tiers. The Action Harness rejects out-of-scope calls.

## Data Layer Analyst

| Available operations                   | Tier scope      |
|----------------------------------------|-----------------|
| <code>get_time_series</code>           | database, cache |
| <code>get_summary_statistics</code>    | database, cache |
| <code>get_time_pattern</code>          | database, cache |
| <code>detect_threshold_breaches</code> | database, cache |
| <code>get_metric_distribution</code>   | database, cache |
| <code>get_business_context</code>      | (no tier)       |
| <code>get_sla_target</code>            | (no tier)       |
| <code>get_configuration</code>         | database, cache |
| <code>get_monthly_cost</code>          | (no tier)       |
| <code>get_top_queries</code>           | (no tier)       |
| <code>get_top_cache_keys</code>        | (no tier)       |
| <code>get_before_after_evidence</code> | (no tier)       |

The Data Layer Analyst covers both the database tier and the cache tier when present, because cache behavior is most meaningfully analyzed in relation to database load.

## Network Analyst

| Available operations                   | Tier scope |
|----------------------------------------|------------|
| <code>get_time_series</code>           | network    |
| <code>get_summary_statistics</code>    | network    |
| <code>get_time_pattern</code>          | network    |
| <code>detect_threshold_breaches</code> | network    |
| <code>get_metric_distribution</code>   | network    |
| <code>get_business_context</code>      | (no tier)  |

| Available operations                   | Tier scope |
|----------------------------------------|------------|
| <code>get_sla_target</code>            | (no tier)  |
| <code>get_configuration</code>         | network    |
| <code>get_monthly_cost</code>          | (no tier)  |
| <code>get_before_after_evidence</code> | (no tier)  |

## Cross-Tier Evaluator

The Evaluator gets the full read surface, including the two cross-tier operations:

| Available operations                  | Tier scope    |
|---------------------------------------|---------------|
| All telemetry reads                   | all tiers     |
| All sidecar reads                     | all tiers     |
| <code>get_correlation_evidence</code> | any tier pair |
| <code>get_pattern_alignment</code>    | any tier pair |

The Evaluator does not normally re-derive specialist-level observations from telemetry. Its primary inputs are the specialist findings (which already cite their evidence). It uses the cross-tier operations to verify cross-tier interactions that no individual specialist could see, and it uses the broader read surface when drift-checking a specialist's reasoning against the underlying data.

## System Mapper

The System Mapper has a distinct surface for Terraform parsing (not part of the §2.6 read contract). It reads:

- Terraform files (HCL) from the bundle's `terraform/` directory.
- `get_configuration(tier)` for each tier, to cross-reference Terraform-declared resources with sidecar-declared configuration.

## Supervisor

The Supervisor does not directly invoke read operations. Its tool surface is the agent-invocation surface: invoke a specialist, request the architecture model, hand off to the Evaluator. These are not read operations; they are workflow operations.

## 4.3 Why structural scoping matters

The partitioning above is not a politeness convention. It is what makes specialist independence a real property:

**A specialist cannot conclude something based on evidence it was structurally prevented from accessing.** The audit trail records every tool call, and the Action Harness rejects out-of-scope calls before they happen. A Compute Analyst that wanted to "peek at" database connection counts to support a compute recommendation simply cannot — the call would be rejected and logged as a violation.

**Specialist independence is what makes the Evaluator's drift-check meaningful.** Three specialists that have no access to each other's tier data, and no access to each other's reasoning, cannot have correlated within the analysis. When the Evaluator then synthesizes across them, the cross-tier interactions it identifies are genuine emergent properties of the application, not artifacts of specialist contamination.

**It also makes audit trail interpretation honest.** A reviewer reading the audit trail can see exactly what each agent saw. There is no "the specialist may have considered X" ambiguity — either X was in the specialist's scope and they read it, or it was out of scope and they could not.

## 4.4 The ReAct loop, in terms of this surface

The ReAct loop diagram in `../ARCHITECTURE.md` §4 is concrete in terms of these operations. A typical specialist ReAct cycle might be:

- **Thought:** "Is there evidence of CPU utilization being chronically below healthy ranges?"
- **Action:** `get_summary_statistics(tier='compute', metric='cpu_p50', window='full')`
- **Observation:** `{p50: 18.4, p90: 24.7, p95: 27.1, mean: 19.2, ...}`
- **Thought:** "The numbers are below healthy compute baseline (40-60% p50). Need to check if this is consistent across days or driven by an outlier."
- **Action:** `get_time_pattern(tier='compute', metric='cpu_p50', granularity='hour')`
- **Observation:** `{pattern: 'flat', weekday_vs_weekend_ratio: 1.02, ...}`
- **Thought:** "Flat low utilization. Need to check current configuration to know what to recommend."
- **Action:** `get_configuration(tier='compute')`
- **Observation:** `{instance_class: 't3.large', instance_count: 8, scaling_policy: 'none'}`
- **Thought:** "Recommend downsize. Need before/after evidence to ground the specific change."
- **Action:** `get_before_after_evidence()`
- **Observation:** `{config: 't3.large × 8 fixed', change: 't3.medium × 4 fixed', outcome: 'CPU p50 stayed 12-22%, SLA preserved, 62% cost reduction'}`
- **Finding:** `issue_found`, recommendation: "Switch t3.large → t3.medium and reduce 8 → 4 replicas", with evidence refs to all four observations above.

Every observation in this loop is logged. The audit trail records the specialist's investigative path, not just the conclusion.

## 4.5 What the tool surface is *not*

- **Not the agent's only context.** The specialist receives application context (business context, SLA target, configuration summary) at invocation time. The tool surface is for *evidence gathering*, not for basic situational awareness.



- **Not a database query language.** The operations are deliberately coarse. A specialist that wants raw rows uses `get_time_series`; a specialist that wants aggregates uses `get_summary_statistics`. Operations that would invite ad-hoc query construction are deliberately absent — they would make audit-trail interpretation harder.
- **Not extensible by the agent.** The surface is fixed. An agent cannot define new operations at runtime. This is what makes the dataset's read contract a real contract.

## 6. Confidence and Drift

---

The Reasoning Harness produces two levels of confidence — specialist-level and evaluator-level — and the Cross-Tier Evaluator performs a drift-check on each specialist's reasoning before synthesizing. These two mechanisms together are the system's defense against single-specialist hallucination.

This document explains what each confidence level measures, how the drift-check works, and what the system honestly cannot catch.

---

### 5.1 Two levels of confidence

A single composite confidence score conflates two genuinely different concerns. A specialist working within its own tier can measure how strong its evidence is and how clear its pattern is, but it cannot measure cross-tier consistency. The Evaluator can measure cross-tier consistency, but it should not re-derive specialist-level confidence — that would defeat the point of having specialists.

The system therefore produces two confidence scores at different layers, measuring different things.

#### Specialist-level confidence

Produced by each Tier Specialist as part of its structured finding. Combines two signals, both measurable within the specialist's own tier:

- **Evidence sufficiency.** How many evidence references support the conclusion, and how well they cover the relevant time windows. A finding citing one tool-call observation is weaker than one citing four observations from independent angles. The Reasoning Harness enforces a minimum count before any `issue_found` can be emitted.
- **Within-tier pattern strength.** For findings that depend on a recurring pattern — daily spikes, business-hours load, sustained underutilization — how well the pattern holds. A pattern that holds on 11 of 14 days (the threshold from the dataset's "11 of 14" rule) is unambiguous; one that holds on 7 of 14 is suggestive but not strong.

The specialist confidence is the product of these two signals, each normalized to a 0–1 scale. The full breakdown is logged to the audit trail so a reviewer can see which sub-signal drove the score.

#### Evaluator-level confidence

Produced by the Cross-Tier Evaluator as part of the synthesized recommendation. Combines three signals:

- **Aggregated specialist confidence.** The average (or minimum, depending on policy — see §5.4) of the specialist-level confidences contributing to this synthesis. A synthesis built on three low-confidence findings cannot itself be high-confidence.
- **Cross-tier consistency.** For scenarios where multiple specialists found related issues, how well their findings align. If the Compute Analyst implicates the database and the Data Layer Analyst confirms a database bottleneck, consistency is high. If they contradict, consistency is low and the Evaluator must reason through the conflict before synthesizing.

- **Trade-off resolution clarity.** How decisively the cost / performance / reliability trade-off was resolvable from the evidence. A recommendation where all three dimensions point the same way (right-size: cheaper, no perf impact, no reliability impact) has high clarity. A recommendation where dimensions conflict (reduce capacity: cheaper, but reliability risk) has lower clarity, and the Evaluator must articulate the trade-off rather than collapse it.

Like specialist confidence, evaluator confidence is logged with its full breakdown.

---

## 5.2 Why this split matters

The two-level design is not bureaucracy. It serves three purposes:

**Honest scoping.** Specialists measure what specialists can measure. The Evaluator measures what only a synthesis layer can measure. Neither pretends to measure the other.

**Drift-check input.** The Evaluator's drift-check (next section) needs to compare specialist-level confidence to its own assessment of specialist reasoning quality. If a specialist reports high confidence but the Evaluator's drift-check finds the reasoning weak, that's a signal. If a specialist reports low confidence and the Evaluator's drift-check agrees, the Evaluator can either request a retry through the Supervisor or surface the low-confidence finding with appropriate framing in the review packet.

**HITL transparency.** The human reviewer sees both levels of confidence in the review packet, with the full breakdown. They can ask: "Was this synthesis confident because the specialists were confident, or because the cross-tier picture was unambiguous?" The answer is in the breakdown.

---

## 5.3 The Evaluator's drift-check

For each specialist finding, the Evaluator performs an explicit drift-check before incorporating the finding into the synthesis. The drift-check asks three questions:

### Q1. Does the recommendation follow from the cited evidence?

The Evaluator examines each evidence reference in the specialist's finding and asks whether the cited observation actually supports the conclusion drawn. A specialist that cites "CPU p50 18-22%" as evidence for "downsize compute by 50%" is making a justifiable inference. A specialist that cites the same evidence as justification for "add a read replica" is reaching beyond what the evidence supports.

### Q2. Is the evidence-binding tight, or does the recommendation reach beyond what the evidence supports?

Some specialists may produce technically-correct recommendations but with weak or partial evidence chains. The Evaluator distinguishes "well-evidenced" from "directionally correct but under-evidenced." Both can be incorporated into synthesis, but the latter weights the recommendation less heavily and surfaces a flag in the review packet.

### Q3. Are there internal contradictions within the reasoning trace?

A specialist whose ReAct trace shows "observe X, conclude Y" — but X and Y are logically inconsistent — has drifted. This is the strongest drift signal. Contradictory specialists are flagged in the review packet and their findings are de-weighted in the synthesis.

Each drift-check produces a verdict per specialist: `tight`, `loose`, or `contradictory`. All three verdicts are logged to the audit trail.

### Why drift-check is the Evaluator's first step, not its last

The Evaluator runs drift-check *before* identifying cross-tier interactions and *before* synthesizing. The order matters: a contradictory specialist finding should not contribute to cross-tier reasoning, and a loose finding should be visible as such before the synthesis combines it with other findings. Running drift-check last would mean the synthesis already happened before drift was noticed.

---

## 5.4 Policy choices that need to be tuned during build

Two policy choices are left intentionally tunable rather than hardcoded:

**Aggregation policy for specialist confidence.** Should the Evaluator average specialist confidences, take the minimum, or weight by drift-check verdict? Each has a defensible rationale. The build phase will tune this against the 18-scenario eval set — see `07-evaluation-methodology.md`.

**Confidence thresholds.** Below what confidence does the Supervisor retry a specialist? Below what evaluator confidence does the recommendation get flagged as low-confidence in the review packet? These thresholds are configuration, not constants, and will be tuned during build.

Both choices are called out as configuration to make them visible and tunable rather than hidden in code. The senior signal is recognizing them as choices worth surfacing.

---

## 5.5 What drift-check does *not* catch

The Evaluator's drift-check is a real defense against single-specialist drift. It is honestly limited in two ways:

**Correlated multi-specialist drift.** When all three specialists drift in the same direction with no internal conflict between them, the Evaluator has no signal to flag. Three specialists that all over-recommend cost cuts will look consistent to the Evaluator because they *are* consistent — just consistently wrong. This class of failure is structurally caught by HITL review: the human, armed with the audit trail and their own judgment, may notice what the Evaluator could not.

**Evaluator drift.** The Evaluator can hallucinate too. Its synthesis can drift away from what the specialist findings support. The mitigation is the audit trail: the Evaluator's synthesis cites which specialist findings supported it, and the human reviewer can spot Evaluator drift by walking the citation chain. The audit trail is the last line of defense for the system's own reasoning quality.

**Evidence that is itself wrong.** The Reasoning Harness enforces evidence-binding, but it cannot verify that the evidence itself is correct. If a tool call returns a wrong observation, every downstream conclusion based on it inherits the error. The Input Harness mitigates this at ingest by validating that data is well-formed and complete, but semantic correctness of the data is the dataset generator's responsibility, not the consumer's.

Naming these limits honestly is part of the architectural signal. The system is not omniscient; it is structurally trustworthy enough that its failures, when they happen, are surfaceable and auditable.

## 5.6 How this shows up in the review packet

The human reviewer sees, for every recommendation:

- The evaluator-level confidence with its breakdown (aggregated specialist confidence, cross-tier consistency, trade-off resolution clarity).
- For each contributing specialist: the specialist-level confidence with its breakdown, and the Evaluator's drift-check verdict.
- The full reasoning chain that justifies the recommendation, with each evidence reference resolvable in the audit trail.

A reviewer skeptical of a recommendation can drill into the chain. A reviewer who trusts the synthesis can approve it quickly. Both behaviors are supported by the same data.

This is what "the audit trail is the visual hero" means in practice. The trail is not a debugging artifact; it is the substrate the human uses to make a decision.

## 7. Persistent Action Record

---

The Persistent Action Record is the system's audit trail. It is treated as its own architectural component rather than as logging bolted onto the agents, because it is fundamentally different code from the agents themselves — structured persistent storage with strong schema discipline, not agentic orchestration.

This document explains what the audit trail captures, how it supports replay, why the storage choice is relational rather than vector, and what the schema looks like in conceptual form.

---

### 6.1 What the audit trail captures

Every significant event in a review cycle is a record. The audit trail is rich enough that a reviewer reading it should be able to reconstruct exactly what the system did and why.

The conceptual record categories:

**Trigger and ingest records.** When a review was requested, by what trigger, against which application, with what bundle hash. The Input Harness's validation outcomes.

**System Mapper records.** The architecture model produced for this review (tiers, dependencies, analysis plan). Any parsing diagnostics from Terraform.

**Supervisor decision records.** Which specialists were invoked and why. Low-confidence handling decisions. Retry decisions. Escalations.

**Specialist ReAct step records.** For each specialist, every cycle of the ReAct loop: the thought, the action (tool call with parameters), the observation (tool result). Each step is its own record, ordered.

**Specialist finding records.** The structured output from each specialist: finding\_type, recommendation (if any), evidence\_refs, reasoning\_trace summary, specialist confidence with breakdown.

**Evaluator records.** Drift-check verdicts per specialist. Cross-tier interactions identified. Trade-off scores across cost / performance / reliability. Final synthesized recommendation. Evaluator confidence with breakdown.

**Action Harness records.** The recommendation gate verdict. The severity classification. The duplication check result.

**Review packet records.** The full review packet as surfaced to the human reviewer. (Stored as the canonical artifact, not just a pointer to it.)

**HITL records.** The human's decision (approve / reject / defer), timestamp, any notes the reviewer attached.

Every record carries:

- A stable unique identifier.
- A timestamp.
- The agent or harness that emitted it.
- The review-cycle identifier (so all records from one cycle can be retrieved together).
- Foreign-key references to upstream records in the causal chain.

The foreign keys are what make the trail navigable: from a final HITL approval, the reader can walk backward through Action Harness gate → review packet → Evaluator synthesis → drift-check verdicts → specialist findings → specialist ReAct steps → tool calls → ingest records.

---

## 6.2 Append-only by design

Records are never updated or deleted. This is the property that makes "replayable" real rather than aspirational.

If a recommendation needs to be corrected — for example, the human reviewer rejects it and provides feedback — the correction is a *new* record that references the original. The historical state at any past point in time is exactly the records that existed up to that point. Nothing is rewritten retroactively.

This discipline matters because the alternative (mutable records) makes the audit trail unreliable for governance purposes. A governance reviewer asking "what did the system recommend three months ago?" needs an answer that does not depend on whether anyone updated the record since.

In storage terms, append-only is enforced both by schema discipline (no `UPDATE` paths in the data access layer) and by application discipline (corrections produce new records). For a portfolio implementation in SQLite, this is straightforward; for a production deployment in Postgres, the same pattern applies.

---

## 6.3 Replayability

A review is **replayable** if, given the review-cycle identifier, the full reasoning chain can be reconstructed deterministically from the audit trail.

What this means in practice:

- Every recommendation has explicit citations to the Evaluator records that produced it.
- Every Evaluator record has explicit citations to the specialist findings that informed it.
- Every specialist finding has explicit citations to the specialist ReAct steps that led to it.
- Every ReAct step has explicit citations to the tool calls it made and the observations it received.
- Every tool call has the bundle hash and the exact parameters used, so the same call against the same bundle produces the same observation.

Given all of this, a developer or reviewer can walk the chain top-down or bottom-up. The system's reasoning is not a black box; it is a navigable graph.

There is one practical caveat: LLM calls are not deterministic at non-zero temperature. The audit trail captures the *outputs* of every LLM call, so the chain is replayable as a record of what happened. Reproducing the same outputs by re-running the LLM is not guaranteed. This is honest behavior, not a flaw — the audit trail is for understanding what happened, not for re-deriving it from scratch.

A `scenario_replay` CLI tool is part of the deliverable plan (see `../BUILD_PLAN.md`). Given a scenario ID and an audit-trail entry, it walks the chain and renders the full reasoning history.

---

## 6.4 Why relational, not vector

A vector database was considered for the audit trail and rejected. The reasoning matters for the design-decisions narrative — see `09-design-decisions.md` for the long-form version. Briefly:

The audit trail's access patterns are relational:

- Append-only writes of structured records.
- Foreign-key traversal to reconstruct causal chains.
- Structured queries ("show all recommendations rejected by HITL in the last 30 days," "show all reviews where drift-check flagged a specialist").
- Deterministic replay from a known starting point.

None of these are similarity-search patterns. A vector database would turn deterministic replay into approximate similarity, drop the foreign-key guarantees, and provide nothing the audit trail needs that relational storage does not already give.

A vector database *would* be appropriate for a different concern — semantic retrieval over past reasoning traces, where an agent wants to find scenarios similar to the current one. That is a separate concern from the audit trail, and it is not currently in scope for this project. If it were, it would be a parallel storage system, not a replacement for the relational audit trail.

For the portfolio implementation, **SQLite** is the chosen engine. Single file, zero infrastructure, ships with Python. The audit trail demo runs anywhere. The senior signal is the *schema design*, not the database engine — the same schema upgrades cleanly to Postgres for a production deployment.

## 6.5 Conceptual schema

The conceptual schema below describes the record categories and their relationships. Concrete column definitions are deferred to implementation.

```
reviews
- review_id (primary key)
- application_id
- trigger_type
- bundle_hash
- started_at
- completed_at
- final_status    (approved | rejected | deferred | error)

ingest_validations
- validation_id
- review_id (FK → reviews)
- check_name
- verdict
- details
- emitted_at

architecture_models
- model_id
- review_id (FK → reviews)
- tiers          (structured: list of tier descriptors)
- dependencies    (structured: list of inter-tier edges)
- analysis_plan   (structured: specialists required, cross-tier pairs)
- emitted_at
```



```

supervisor_decisions
- decision_id
- review_id (FK → reviews)
- decision_type      (invoke_specialist | retry | escalate | aggregate)
- decision_details   (structured)
- emitted_at

```

```

specialist_steps
- step_id
- review_id (FK → reviews)
- specialist      (compute | data_layer | network)
- step_index      (ordinal within the specialist's ReAct loop)
- thought
- action_name
- action_params   (structured)
- observation      (structured)
- emitted_at

```

```

specialist_findings
- finding_id
- review_id (FK → reviews)
- specialist
- finding_type      (issue_found | no_issue_found | insufficient_data)
- recommendation     (nullable)
- evidence_refs      (structured: list of step_id references)
- reasoning_trace_summary
- specialist_confidence
- confidence_breakdown (structured)
- emitted_at

```

```

evaluator_drift_checks
- drift_check_id
- review_id (FK → reviews)
- target_finding_id (FK → specialist_findings)
- verdict          (tight | loose | contradictory)
- reasoning
- emitted_at

```

```

evaluator_records
- evaluator_id
- review_id (FK → reviews)
- cross_tier_interactions (structured)
- trade_off_scores         (structured: cost, performance, reliability)
- synthesis                 (structured: balanced recommendation)
- evaluator_confidence
- confidence_breakdown      (structured)
- contributing_findings      (structured: list of finding_id references)
- emitted_at

```

```

action_harness_gate_records
- gate_id
- review_id (FK → reviews)
- evaluator_id (FK → evaluator_records)
- well_formedness_verdict
- evidence_completeness_verdict
- severity_classification
- duplication_check_result
- overall_verdict          (pass | flagged | rejected)
- emitted_at

```

```

review_packets
- packet_id
- review_id (FK → reviews)
- gate_id (FK → action_harness_gate_records)
- packet_contents      (canonical JSON of what the human sees)
- emitted_at

hitl_decisions
- decision_id
- review_id (FK → reviews)
- packet_id (FK → review_packets)
- decision      (approve | reject | defer)
- reviewer_notes
- decided_at

```

The schema is normalized enough to support structured queries but flat enough to be readable in a SQLite shell. The "(structured)" annotations indicate fields stored as JSON within a column — pragmatic for SQLite, easy to query with `json_extract`, and clean to migrate to Postgres `jsonb` or to fully-typed columns later.

## 6.6 What the audit trail does *not* do

- **It is not an event bus.** Agents do not communicate by writing to the audit trail and reading each other's writes. Agent-to-agent communication is the Supervisor's responsibility. The audit trail is a parallel write-only stream.
- **It is not a knowledge base.** The agents do not query the audit trail as part of their reasoning. They reason against the bundle. The audit trail exists for human-facing observability and replay, not for agent memory.
- **It is not a metrics store.** Operational metrics (latency, error rates, LLM token counts) may flow into a separate observability stack. The audit trail is for governance-facing reasoning traceability, which is a different artifact with different requirements.

Conflating any of these with the audit trail's purpose would dilute it. Keeping its scope clean is part of the architectural signal.

## 6.7 The audit trail as the README's visual hero

The original Project 2 framing called out the Persistent Action Record as the visual hero of the README. That carries forward here.

The strongest single section of the README is the **audit-trail walkthrough**: pick one scenario, run it through the system, and show every record that was written, in causal order, with the recommendation at the end traceable back through the entire chain. A hiring manager who reads only that section should understand what the project does and why it is worth the engineering depth.

This is what "the audit trail is the artifact a governance reviewer engages with" means in concrete terms. The walkthrough is not a debugging tool. It is the system's primary deliverable to anyone who needs to *understand* a recommendation rather than just consume it.

## 8. Evaluation Methodology

The system is evaluated against the 18-scenario synthetic dataset specified in the Cloud Governance Dataset Generation Plan v3. This document defines what "the system passes" actually means — how agent output is scored against the target recommendation each scenario carries.

Without an evaluation methodology, the demo has no objective success criterion. Specifying one is part of what makes this a portfolio project rather than a feature showcase.

### 7.1 Two evaluation layers, distinct concerns

There are two evaluation systems operating in different layers:

| Layer            | What it evaluates                                                                                                                           | Where it lives                                                   |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Dataset QA       | Whether the synthetic dataset itself is well-formed (record counts, timestamp continuity, sidecar field presence, correlation timing, etc.) | Dataset Generation Plan v3 §8 — runs as part of dataset pipeline |
| Agent evaluation | Whether the agent system's recommendation matches the scenario's target recommendation in substance                                         | This document — runs against produced reviews                    |

The dataset QA validates the inputs. The agent evaluation validates the outputs. They are independent and should be kept that way.

### 7.2 The agent evaluation rubric

For each scenario, the agent system produces a review packet containing a recommendation. The evaluation compares that recommendation against the scenario's target rich recommendation across five rubric dimensions:

#### R1 — Correct issue identification

**Question:** Did the agent identify the correct issue (or correctly identify that there is no issue)?

**Scoring:**

- **Match:** The agent's `finding_type` and identified problem match the scenario's target.
- **Partial:** The agent identified a related issue but missed the primary one (e.g. identified compute over-provisioning when the actual issue was database over-provisioning).
- **Miss:** The agent identified a different issue or invented a non-existent one.

**Notes:** For healthy scenarios (Scenario 6) and mostly-healthy scenarios (18), a `no_issue_found` or minimal-recommendation outcome is the correct answer. The Restraint scenarios (6, 16, 18) explicitly test

whether the agent over-recommends.

## R2 — Correct recommendation direction

**Question:** Did the agent recommend an action in the correct direction?

**Scoring:**

- **Match:** Direction is correct (e.g. downsize when target is downsize; add replicas when target is add replicas).
- **Partial:** Direction is partially correct (e.g. agent recommends right-sizing both compute and database, target was only database — direction is right for one tier and wrong for the other).
- **Miss:** Direction is wrong (e.g. agent recommends scale up when target is scale down).

**Notes:** This dimension catches sign errors in reasoning that R1 might miss.

## R3 — Recommendation specificity

**Question:** Did the agent's specific quantities (instance classes, replica counts, percentage savings, pool sizes) match the target within an acceptable tolerance?

**Scoring:**

- **Match:** Specific values match the target exactly or within a small tolerance (configurable per scenario).
- **Partial:** Some specifics match, others do not.
- **Generic:** The agent produced a recommendation in the correct direction but without the specific quantities the target carries (e.g. "downsize compute" instead of "switch from t3.large × 8 to t3.medium × 4").

**Notes:** Generic recommendations score lower than matching specifics. This dimension measures whether the agent successfully grounded its recommendation in the sidecar metadata and before/after evidence — the Specificity Grounding evaluation property from the dataset doc.

## R4 — Appropriate restraint or deferral

**Question:** For scenarios that demand restraint (6, 16, 18) or diagnostic deferral (17), did the agent correctly produce a "no action," "minor adjustment only," or "recommend further investigation" outcome?

**Scoring:** Pass / fail. This dimension is binary because it tests a specific behavior that the agent either exhibited or did not.

**Notes:** This is one of the most important dimensions for this system. Most agent systems are biased toward producing recommendations. The system's ability to *not* recommend when the data does not justify recommendation is what distinguishes it from a simple optimization tool.

## R5 — Evidence chain integrity

**Question:** Does the agent's recommendation cite evidence that actually resolves to logged audit-trail records?

**Scoring:**

- **Pass:** All evidence references in the recommendation resolve to records in the audit trail.
- **Partial:** Most evidence references resolve, but some are missing or refer to records that do not exist.
- **Fail:** Significant evidence references are missing or do not resolve.

**Notes:** This dimension is the structural test for the Reasoning Harness's evidence-binding enforcement. A score below pass here is a system bug, not an agent reasoning failure.

## 7.3 LLM-as-judge for the rubric

R1, R2, and R3 are semantic comparisons between produced and target recommendations. They require LLM-as-judge to evaluate fairly — a string-equality check would miss cases where the agent produced a valid recommendation phrased differently from the target.

R4 is partially structural (was a non-action outcome produced?) and partially semantic (was the deferral framed correctly?). LLM-as-judge handles the semantic part; the structural part is deterministic.

R5 is fully structural and runs without LLM-as-judge — it walks the audit trail and verifies references.

The LLM-as-judge for evaluation should be a strong model (Claude Sonnet or stronger) running with a fixed rubric prompt that includes the target recommendation, the produced recommendation, and the scoring scale for each dimension. The judge prompt is itself a deliverable; it should be authored and versioned alongside the eval set.

## 7.4 Per-scenario scoring and aggregate scoring

Per-scenario: each of R1 through R5 produces a score on its own scale. A scenario's score is reported as a vector across the five dimensions, not collapsed into a single number. Collapsing would hide useful signal — a scenario where R1 matches but R3 is generic tells you something different from a scenario where R3 matches but R4 fails.

Aggregate (across 18 scenarios): the report shows per-dimension distributions. "Across 18 scenarios, R1 matched in 14, partial in 3, missed in 1." This is the format that surfaces patterns — for example, that the system consistently struggles with restraint scenarios, or that specificity scoring is uneven across tiers.

A single composite "X% passed" number is intentionally not produced. It would be misleading.

## 7.5 Tolerance bands for numeric matching

R3 (specificity) requires tolerance bands for numeric quantities, because the agent may produce slightly different numbers than the target while still being substantively correct.

Suggested defaults (tunable):

- **Instance class:** exact match required. (t3.large ≠ t3.medium for scoring purposes.)
- **Instance count:** ±1 unit acceptable for partial credit; exact match for full.
- **Replica count:** exact match required.
- **Percentage savings:** ±5 percentage points acceptable for partial credit; ±2 points for full.
- **Dollar amounts:** ±10% acceptable for partial credit; ±5% for full.
- **Pool sizes:** ±25% acceptable for partial credit; ±10% for full.
- **Latency targets:** ±20% acceptable for partial credit; ±10% for full.

These are starting points. They should be revisited after the first eval run.

---

## 7.6 The three subset modes

Running the full 18 scenarios produces a comprehensive picture but is expensive (see `08-llm-and-cost.md`). Three subset modes serve different purposes:

**Smoke test (3 scenarios).** Scenarios 1, 7, 6 — one single-tier negative, one cross-tier negative, one healthy. Confirms the pipeline runs end-to-end. Useful for rapid iteration during development.

**Coverage subset (6 scenarios).** Scenarios 1, 4, 7, 9, 12, 17 — one of each major scenario type (single-tier compute, single-tier database, cross-tier cache, cross-tier scheduled-scaling, mixed-state, diagnostic deferral). Useful for catching regressions across architectural patterns without running the full eval.

**Full eval (18 scenarios).** All scenarios. Run before any meaningful release or after any change that could plausibly affect multiple scenarios.

The smoke test should complete in minutes. The full eval may take longer depending on LLM latency and ReAct loop depth.

---

## 7.7 What the evaluation methodology does *not* do

- **It does not measure latency or throughput.** Operational performance is a separate concern, captured in observability metrics outside the audit trail.
  - **It does not evaluate the audit trail's correctness directly.** The audit trail's correctness is structurally enforced by the Reasoning Harness's evidence-binding; R5 verifies the chain integrity but does not re-derive the chain's content.
  - **It does not score the agent against itself across runs.** Non-determinism in LLM output means cross-run comparisons are noisy. The methodology compares agent output to target, not to prior agent runs.
  - **It does not gate releases.** This is a portfolio project; there is no release gate. In a production deployment, the methodology would inform a release gate, but that is out of scope.
- 

## 7.8 The eval set as a build artifact

The eval set — the 18 scenarios with their target recommendations, the rubric, the LLM-as-judge prompt, and the tolerance bands — is itself a deliverable. It should live in the repository alongside the agent code, versioned with the agent, and runnable on demand.

A `make eval` entry point (or equivalent) runs the full eval and produces a structured report. The report is itself an artifact that can be inspected, archived, and compared across versions of the agent.

This closes the loop on "what does it mean for this system to work?" — there is a concrete answer, and it is reproducible.

## 9. LLM Provider Mix and Cost

---

The system uses two tiers of LLM capability — a fast, cheap model for the Tier Specialists, and a stronger model for the Cross-Tier Evaluator. This document explains the choice, estimates the cost, and describes how the three subset modes (smoke, coverage, full) keep iteration affordable.

---

### 8.1 The two-tier model split

| Layer                | Model                | Rationale                                                                                                                                                                                                                                             |
|----------------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tier Specialists     | <b>Claude Haiku</b>  | Specialists run ReAct loops with many small reasoning turns. Fast inference and low per-call cost matter more than peak reasoning capability. Haiku handles structured reasoning against a constrained tool surface well.                             |
| Cross-Tier Evaluator | <b>Claude Sonnet</b> | The Evaluator does the hardest single-call reasoning in the system — drift-checking each specialist, identifying cross-tier interactions, and synthesizing a balanced recommendation with trade-off scoring. Stronger capability earns its cost here. |
| LLM-as-judge (eval)  | <b>Claude Sonnet</b> | The evaluation rubric requires nuanced semantic comparison between produced and target recommendations. Sonnet handles this well. (Not part of the agent system itself; only used at eval time.)                                                      |

This is a single-provider design. One API key, one SDK, one rate-limit story, one billing line. The choice avoids the operational overhead of mixing providers without sacrificing capability.

---

### 8.2 Why not a single model throughout

Two alternatives were considered:

**All Haiku.** Cheaper, faster across the board. The Evaluator's job is hard enough that a Haiku-only design risks weak synthesis, especially on trade-off scenarios where the model needs to articulate competing dimensions. Not a hard ceiling for the portfolio implementation, but the architectural story is weaker — "I used the same cheap model for everything" leaves cost-aware design unexamined.

**All Sonnet.** Stronger across the board, but specialists do many ReAct cycles where stronger reasoning is not the constraint. Costs add up quickly without proportional capability gain. Wastes capability on the wrong layer.

The two-tier split is itself a design signal. It demonstrates that the architect thought about *where* capability matters, not just *whether* to use a strong model. This is the kind of decision a senior reviewer notices.

## 8.3 Cost estimation per review

Per-review cost depends on the scenario complexity, but a representative estimate for one full review cycle:

| Component                              | Calls                   | Notes                                                                     |
|----------------------------------------|-------------------------|---------------------------------------------------------------------------|
| Input Harness validation               | 0 LLM calls             | Pure schema validation.                                                   |
| Supervisor decisions                   | 1–2 LLM calls (Haiku)   | Light orchestration reasoning.                                            |
| System Mapper                          | 1 LLM call (Haiku)      | Architecture model generation after Terraform parse.                      |
| Tier Specialist ReAct (per specialist) | 5–10 LLM calls (Haiku)  | Depends on how many ReAct cycles the specialist needs.                    |
| Tier Specialists (all three)           | 15–30 LLM calls (Haiku) | Three specialists × 5–10 cycles each.                                     |
| Cross-Tier Evaluator                   | 3 LLM calls (Sonnet)    | Drift-check, cross-tier identification, synthesis.                        |
| Action Harness gate                    | 0–1 LLM call (Haiku)    | Structural checks are deterministic; severity classification may use LLM. |
| <b>Total per review</b>                | <b>20–37 LLM calls</b>  | Mix of Haiku and Sonnet.                                                  |

At current Anthropic pricing (subject to change), a single review costs in the order of cents to low single-digit dollars. Across 18 scenarios:

| Mode            | Reviews      | Approximate scale                         |
|-----------------|--------------|-------------------------------------------|
| Smoke test      | 3 scenarios  | Small change, run frequently.             |
| Coverage subset | 6 scenarios  | 2× smoke; run before significant changes. |
| Full eval       | 18 scenarios | 3× coverage; run before milestones.       |

The exact numbers are not the point — what matters is that the system is affordable to iterate on. A full eval is not free, but it is not a meaningful budget concern for a portfolio project.

For accurate up-to-date pricing, see Anthropic's pricing page. The numbers above assume the model versions current at the start of the build.

## 8.4 Cost-aware design patterns in the architecture



Beyond the two-tier model split, several architectural choices reduce LLM cost:

**Parallel specialist execution.** The three specialists run in parallel against independent tool surfaces. This does not reduce total LLM calls, but it reduces wall-clock latency, which is a usability win for the demo.

**Supervisor early-skip.** When the System Mapper's analysis plan indicates a specialist is not needed for this application's topology, the Supervisor skips that specialist. Scenarios that only involve compute (1, 2, 5, 16) do not invoke the Data Layer or Network Analysts.

**Healthy-application short-circuit (consideration).** For applications where the Supervisor can establish that all tiers are clearly healthy (Scenario 6), the full ReAct loop is unnecessary — a confident `no_issue_found` from a lightweight check could skip the specialists entirely. This is a tunable optimization, not a built-in. Worth experimenting with during build to see how much cost it saves.

**Deterministic Action Harness checks.** Recommendation gate checks (well-formedness, evidence completeness, duplication) are structural and run without LLM. Severity classification may use LLM but can fall back to a rules-based classifier for cost.

**Caching the Evaluator's drift-check.** A drift-check is a function of (specialist finding) → verdict. If the same finding is evaluated twice (during retries, for instance), the verdict can be cached. The audit trail's append-only design supports this — caching at the application layer above the audit trail is straightforward.

---

## 8.5 The eval cost is separate from the system cost

Running the evaluation methodology (LLM-as-judge with Sonnet against each produced recommendation) adds cost on top of the agent system's cost. For an 18-scenario full eval, expect roughly:

- 18 reviews × 20–37 LLM calls = the agent-system cost (mostly Haiku, some Sonnet).
- 18 LLM-as-judge calls × 5 rubric dimensions = ~90 Sonnet calls for evaluation, plus a smaller number of structural R5 checks (no LLM).

This is non-trivial but bounded. A reasonable cadence is to run smoke after every commit, coverage subset before merging significant changes, and full eval before each milestone.

---

## 8.6 What if the budget is tighter

If the project ever needs to run on a very tight budget (extended iteration, no Sonnet access, etc.), reasonable degradations:

- **Replace Sonnet with Haiku for the Evaluator.** Accept weaker synthesis on trade-off scenarios. Note this honestly in the eval report.
- **Replace Sonnet LLM-as-judge with deterministic comparison.** Trade rubric fidelity for cost. Some dimensions (R1, R5) degrade cleanly to deterministic checks; R2, R3, R4 do not.
- **Run only the smoke test cadence.** Skip coverage and full eval until budget allows.

None of these are recommended for a portfolio deliverable. They are escape hatches if iteration cost becomes a real constraint.

---

## 8.7 Provider portability

Although the design uses Claude models, the architectural pattern is provider-agnostic. The Tier Specialists need a fast, cheap, structured-output-friendly model; the Evaluator needs a stronger reasoning model. Substituting GPT-4o-mini and GPT-4o (or any equivalent two-tier split from another provider) is a configuration change, not an architectural one.

This is mentioned not as a recommendation to multi-provider, but to note that the choice of Claude is a deliberate single-provider simplification, not a hard architectural dependency.

## 10. Design Decisions and Trade-offs

---

Every meaningful architectural decision in this project has alternatives. This document names each decision, the alternatives that were considered, and why the chosen path was preferred. It is the section a senior reviewer reads to understand *how* the architecture was reasoned about, not just *what* the architecture is.

The ability to articulate these trade-offs is more valuable than the chosen designs themselves.

---

### 9.1 ReAct specialists over zero-shot specialists

**Decision:** Each Tier Specialist runs a ReAct loop against a constrained read surface, producing structured findings with explicit evidence-binding.

**Alternative considered:** Zero-shot specialists. Pass the full bundle (telemetry + sidecar) into a single LLM call per specialist and ask for a structured recommendation back.

**Why ReAct was chosen:**

- **Audit-trail richness.** A zero-shot specialist's audit record is "here was the input, here was the output." There is no investigative trace to review. The Persistent Action Record's value depends on having something to record beyond input/output pairs. ReAct produces a sequence of thought / action / observation steps that constitute genuine reasoning trace.
- **Evidence-binding is enforceable.** With ReAct, evidence references can point to specific tool-call observations logged in the audit trail. A zero-shot recommendation would have to either reference raw telemetry rows (uselessly granular) or reference summary statistics that the LLM internally computed but never explicitly emitted (unverifiable).
- **Resembles how engineers actually investigate.** A senior SRE looking at a multi-tier issue does not absorb all telemetry at once and conclude. They form a hypothesis, query for confirmation, refine. The system's reasoning shape matches this. The audit trail therefore reads like an investigation transcript rather than a black-box decision.
- **Restraint is easier to engineer.** A zero-shot specialist given a healthy application's full bundle may still find something to recommend, because the prompt is asking for a recommendation. A ReAct specialist that hypothesizes, queries, and finds no evidence of a problem can naturally conclude `no_issue_found`. The three-valued `finding_type` fits ReAct's investigative shape better than zero-shot's single-call shape.

**Trade-off accepted:** ReAct costs more — more LLM calls per specialist per review. The cost is acceptable given the system's identity (recommender, not high-throughput automation) and the audit-trail value gained.

**When zero-shot would be the right choice:** A high-throughput recommender where audit trail depth is secondary to per-recommendation latency and cost. That is not this system.

---

### 9.2 Relational storage for the audit trail, not vector

**Decision:** The Persistent Action Record uses SQLite (with a Postgres-compatible schema) for the portfolio implementation.

**Alternative considered:** Vector database for the audit trail.

**Why relational was chosen:**

The audit trail's access patterns are not similarity-based:

- **Append-only writes** of structured records.
- **Foreign-key traversal** from a final recommendation back through the Evaluator synthesis, drift-checks, specialist findings, ReAct steps, and tool calls.
- **Structured queries** like "show all reviews where drift-check flagged a specialist" or "show all recommendations rejected by HITL in the last 30 days."
- **Deterministic replay** from a known starting point.

None of these are vector-database strengths. A vector database would turn deterministic replay into approximate similarity, abandon foreign-key guarantees, and provide nothing the audit trail needs that relational storage does not already give.

**Where vector storage would be appropriate:** A separate concern — semantic retrieval over past reasoning traces, where an agent wants to find scenarios similar to the current one. That is a *memory* system, not an *audit* system. If the project later added agent memory, it would be a parallel storage layer with its own data model, not a replacement for the audit trail. The audit trail and the memory system are different artifacts with different requirements; conflating them would compromise both.

**Trade-off accepted:** Semantic search over the audit trail is not supported. If a reviewer asks "have I seen a recommendation like this before?", they walk the trail by structured query, not by similarity. For a governance audit context, structured queries are the right primitive.

---

## 9.3 Action Harness intentionally narrow

**Decision:** The Action Harness has two scoped jobs (tool-surface control during analysis, quality-gating of the final review packet) and does no actuation.

**Alternative considered:** Expand the Action Harness to include whitelisted actuators — generate JIRA tickets, draft Terraform PRs, send notifications.

**Why narrow was chosen:**

The system's identity is **recommendation under HITL**. Adding actuators would shift the system toward a hybrid recommender-actuator, which is two systems wearing one face. The reasoning required to produce a sound recommendation is different from the safety machinery required to execute one; conflating them dilutes both.

Resisting the temptation to inflate this layer is itself the architectural signal. Many candidate projects would expand the Action Harness to make it "feel bigger." The discipline of keeping it small says something about the architect — that scope was chosen for a reason, not by default.

**Trade-off accepted:** The Action Harness story is less elaborate than it could be. The original Project 2 had a richer Action Harness because its identity included actuation (ServiceNow ticket creation, HITL approval for state-changing operations). This project's narrower Action Harness is a deliberate consequence of identity

choice, not an oversight.

**What the interview answer looks like:** "I scoped the Action Harness to match the system's identity as a recommender. Inflating it would have made it look bigger but would have diluted the recommender story. The Reasoning Harness is the heavy layer in this system, and that is where the architectural work actually lives."

---

## 9.4 Two-tier LLM mix (Haiku + Sonnet) over single-tier

**Decision:** Specialists use Claude Haiku; the Evaluator uses Claude Sonnet.

**Alternatives considered:**

- All Haiku (cheaper, weaker Evaluator synthesis on trade-off scenarios).
- All Sonnet (stronger across the board, wastes capability on specialist ReAct cycles that don't need it).

**Why two-tier was chosen:**

The cost-per-capability ratio is non-uniform across the system. Specialists do many small reasoning turns where speed and cost dominate. The Evaluator does a few large reasoning turns where capability dominates. Matching model strength to workload reduces waste in both directions.

This is also a demonstrable design choice. "I matched model tier to reasoning intensity" is a defensible answer to "why did you choose your LLM mix?" — more defensible than "I used the same model everywhere because it was simpler" or "I used the strongest model because it was best."

**Trade-off accepted:** Two model billing categories instead of one. The operational complexity is negligible (same provider, same SDK), and the cost savings on the high-volume specialist tier are real.

---

## 9.5 Terraform parsing for the System Mapper, not sidecar-only

**Decision:** The System Mapper parses Terraform stubs to derive the tier-dependency graph. The sidecar carries configuration values; Terraform provides structural relationships.

**Alternative considered:** Drop the Terraform parsing step entirely. Read everything from the sidecar.

**Why Terraform was chosen:**

The sidecar's flat configuration (instance class, count, replica count, etc.) does not encode *how the tiers connect*. Terraform's `depends_on` and reference relationships do. Without Terraform, the System Mapper degrades into a JSON deserializer with no analytical job — and its existence as a separate agent becomes hard to justify.

With Terraform, the System Mapper does genuine work: parsing a declarative infrastructure description and producing a structured analysis plan. The Supervisor's decisions about which specialists to invoke have a real input, and the Cross-Tier Evaluator's cross-tier pairs of interest are derived, not invented.

**Trade-off accepted:** Terraform parsing adds a real component to the project (a Python HCL parser, hand-crafted Terraform stubs per scenario). It is non-trivial work, but it is the kind of work that is realistic for a cloud governance system. A reviewer evaluating the project's realism gives credit for parsing real infrastructure description rather than reading from a curated metadata blob.

**What the interview answer looks like:** "The System Mapper parses Terraform because that's where inter-tier dependencies live. The sidecar tells me the tiers exist; Terraform tells me how they connect. Without that, the System Mapper has no real job."

---

## 9.6 Three specialists in parallel, not sequential

**Decision:** The three Tier Specialists run in parallel against independent tool surfaces. Each specialist sees only its own tier's data.

**Alternatives considered:**

- Sequential specialists (one runs, then the next reads the prior specialist's finding before running).
- A single multi-tier specialist that reasons across all three tiers at once.

**Why parallel-and-independent was chosen:**

- **Latency.** Three parallel specialists complete in roughly the time of the slowest one, rather than the sum of all three.
- **Drift-check meaningfulness.** Specialists that see each other's reasoning can correlate within the analysis. Specialists that are structurally prevented from seeing each other's tier data cannot have correlated. The Cross-Tier Evaluator's drift-check is meaningful only when the specialists are independent.
- **Specialist depth.** Each specialist can pursue its tier's investigation without cognitive load from other tiers. The Compute Analyst does not have to reason about cache behavior; the Cache Analyst does not have to reason about network topology.

**Trade-off accepted:** Some scenarios benefit from cross-tier evidence during specialist analysis. Cross-tier reasoning is deferred to the Evaluator. This is a deliberate concentration of cross-tier logic in one place rather than a diffuse cross-tier awareness in every specialist.

---

## 9.7 Two-level confidence model

**Decision:** Specialist-level confidence (within-tier signals) and Evaluator-level confidence (cross-tier signals). Two scores, two layers.

**Alternative considered:** A single composite confidence per recommendation.

**Why two levels were chosen:**

Specialists can measure what specialists can measure (evidence sufficiency, within-tier pattern strength). They cannot honestly measure cross-tier consistency. The Evaluator can measure cross-tier consistency but should not re-derive specialist-level signals — that would defeat the point of specialist independence.

Keeping the two levels separate means each score is honest about what it represents. A reviewer looking at the review packet can see whether the high confidence came from strong specialist evidence or from clear cross-tier alignment — different mechanisms with different implications.

**Trade-off accepted:** The reviewer interface has two confidence scores instead of one. The reviewer must learn to read the breakdown, which is a small cognitive cost in exchange for honest signal.

---

## 9.8 Hand-crafted synthetic dataset, reverse-engineered from targets

**Decision:** The evaluation dataset is 18 hand-crafted scenarios, each reverse-engineered from a target recommendation.

**Alternatives considered:**

- Real cloud telemetry (from a personal AWS account or public dataset).
- Random / generative scenarios (have an LLM produce variations).

**Why hand-crafted reverse-engineered was chosen:**

- **Ground truth.** Each scenario has a known correct recommendation. Real telemetry does not.
- **Coverage of edge cases.** The dataset is designed to exercise restraint scenarios (healthy applications), diagnostic deferral (ambiguous root cause), mixed-state (some tiers healthy, some not), and trade-off scenarios (cost vs reliability). These are the cases that distinguish a sophisticated recommender from a simple one. Real telemetry would not exercise them deterministically.
- **Reproducibility.** A reviewer cloning the repo can run the same eval and get the same scenarios. With real cloud account telemetry, this is not possible.

**Trade-off accepted:** The evaluation is on synthetic data, not real production telemetry. This is acknowledged honestly in [10-limitations.md](#). The dataset doc's reverse-engineering principle is a deliberate evaluation device, not a limitation of the methodology.

---

## 9.9 Recommender-only, no actuation

**Decision:** The system produces recommendations for human review. It does not actuate state changes.

**Alternative considered:** Add actuation for low-risk recommendations (auto-approve cost-down recommendations under a dollar threshold, for example).

**Why recommender-only was chosen:**

See §9.3 for the long version. Short version: identity clarity. A recommender-only system has a coherent architectural story. A hybrid recommender-actuator has two stories that compete for the same architectural surface area.

For a portfolio project, the recommender-only choice also keeps scope tight. Actuation would require external integrations (AWS APIs, Terraform automation, JIRA, etc.) that distract from the architectural story this project is telling.

---

## 9.10 What I deliberately said no to

The architecture is shaped as much by what was rejected as by what was chosen. The most consequential "no":

- **No role-based access filtering.** This was a feature of the original Project 2's IT support context (multiple user personas with different access). This project has a single reviewer per review; role-based filtering would be theater.
- **No production observability stack.** Metrics, traces, logs at the operational layer are out of scope. The audit trail is for reasoning provenance, not operational telemetry.
- **No real-time streaming.** The system is review-based, not stream-based. Reviews are bounded units of work.
- **No agent memory across reviews.** Each review is stateless. The audit trail is for humans to query, not for agents to read.
- **No tool extensibility at runtime.** The ReAct tool surface is fixed by the read contract. Agents cannot define new tools dynamically.
- **No multi-application reasoning.** Each review is scoped to one application. Cross-application patterns (e.g. "this same recommendation pattern appears across five apps") are out of scope.

Each "no" was a real choice, not an oversight. Naming them is what makes the scoping look deliberate.

---

## 9.11 The shape of this section in an interview

A hiring manager who asks "how did you decide [X]?" gets a structured answer:

"I considered [alternative]. I chose [decision] because [reason A], [reason B]. The trade-off is [trade-off].  
The alternative would have been right if [different context]."

Every entry in this document follows that shape. The point is not to defend the chosen design as objectively correct — the point is to show that it was *chosen*, with awareness of its alternatives and its limits. Senior reviewers care more about the reasoning than the conclusion.



# 11. Limitations

---

Naming what a system deliberately does *not* do is part of an honest architecture. This document scopes the project's limits so that no reader is left guessing whether something was overlooked or excluded on purpose.

The limits fall into three groups: identity-driven (the system is a recommender, not an actuator), scope-driven (a portfolio project must end somewhere), and dataset-driven (synthetic data has bounded fidelity).

---

## 10.1 Identity-driven limits

These follow directly from the choice to make the system a recommender under HITL.

### **The system does not actuate state changes.**

It produces recommendations. A human approves, rejects, or defers. Anything downstream of the human's approval — opening a ticket, drafting a Terraform PR, sending notifications, executing API calls — is out of scope. This is a deliberate identity choice, not a scoping oversight. See [01-design-principles.md §1.1](#) and [09-design-decisions.md §9.9](#).

### **There is no rollback model.**

Because the system does not actuate, there is nothing to roll back. The audit trail records what was recommended, but a recommendation that was approved and then proved wrong is the human's call to reverse, not the system's.

### **There is no closed-loop feedback.**

The system does not learn from human approvals or rejections. It does not adjust confidence thresholds, weighting policies, or recommendation patterns based on HITL outcomes. The HITL decision is logged in the audit trail; it is not fed back into the agents.

A closed-loop system would be a genuinely different project. The choice not to build one keeps the recommender story coherent.

### **There is no agent memory across reviews.**

Each review is stateless from the agents' perspective. A specialist analyzing application X today does not know that the same specialist analyzed application Y last week. The audit trail is for humans to query, not for agents to consume.

This is a deliberate scope choice — see [09-design-decisions.md §9.10](#). A memory system would be a separate architectural concern with its own storage layer, its own retrieval surface, and its own consistency story. Conflating it with the audit trail would dilute both.

---

## 10.2 Scope-driven limits

These reflect choices about where a portfolio project ends.

### **No production observability stack.**

The audit trail captures reasoning provenance. It does not capture operational metrics — latency distributions, error rates, LLM token spend, cache hit ratios in the agent runtime itself. A production deployment would integrate with an operational observability stack (Prometheus, OpenTelemetry, etc.); this project does not.

### **Continual monitoring is design intent, not implementation.**

The Supervisor's continual-monitoring story — deciding when applications are due for review based on schedules, recent infrastructure changes, or telemetry anomalies — is described as architectural intent but is implemented only as scenario-batch invocation for the portfolio demo. A real continual-monitoring trigger system would add change-detection logic, schedule management, and anomaly detection, none of which are needed for the architectural story this project tells.

### **No multi-application reasoning.**

Each review is scoped to a single application. The system does not reason across applications — for example, "this same over-provisioning pattern appears across five apps." Pattern aggregation across applications would be a separate analytical layer, out of scope here.

### **No UI in v1.**

The HITL surfacing in v1 produces a review packet as structured data. Rendering it as a usable UI is deferred — see [99 - todo.md](#). The review packet's data contract is fully specified; the UI is a presentation layer that any consumer can implement.

### **Action whitelist is small.**

Recommendations are scoped to a deliberately narrow set: instance right-sizing, replica adjustment, pool sizing, scaling policy change, query/cache optimization, network topology change, SLA review. The system is not designed to handle arbitrary infrastructure recommendations (security posture, IAM policy, encryption strategy, etc.).

This is a scoping choice that protects the project from feature creep. Each scenario in the eval set produces a recommendation within this whitelist. A broader scope would require broader specialists and broader Evaluator capability.

### **Single-domain agents.**

The Tier Specialists are scoped to compute / data layer / network. A real cloud governance system would likely add security, compliance, and data-lifecycle specialists. The architectural pattern supports adding them, but they are out of scope for this implementation.

### **No multi-region or multi-cloud reasoning.**

The system reasons about a single deployment of a single application. Multi-region deployments, multi-cloud strategies, and cross-cloud cost arbitrage are out of scope.

---

## **10.3 Dataset-driven limits**

These reflect honest acknowledgment of where the synthetic dataset's fidelity ends.

## Telemetry is synthesized, not observed.

The 18 scenarios are reverse-engineered from target recommendations. See the Cloud Governance Dataset Generation Plan v3 §1.2. This is a deliberate evaluation device — every scenario has ground truth — but it means the system has not been tested against the messy, ambiguous, partially-broken telemetry that real cloud deployments produce.

The system's robustness to data quality issues (missing intervals, sensor drift, metric label changes, partial outages) is not exercised by the synthetic dataset. The Input Harness handles structural well-formedness; it does not handle semantic data-quality degradation that real telemetry would surface.

## The before/after evidence is fabricated.

Each scenario carries a before/after evidence clause of the form "in other apps with identical configuration X, after change Y, outcome Z was observed." These clauses are authored alongside the scenario; they are not drawn from real engineering case studies. They serve as a grounding device to give the LLM something to anchor specific numeric recommendations to.

A reviewer evaluating real-world applicability should treat the specific numbers (62% cost reduction, \$1,680 monthly savings, latency 650ms → 220ms) as illustrative, not as empirical claims. The numbers are realistic in shape; their specific values are part of the synthetic scaffolding.

## The 18 scenarios are not exhaustive.

The scenario set was designed for coverage of distinct architectural patterns (single-tier negative, cross-tier negative, mixed, healthy, mostly-healthy, diagnostic deferral). It is not a stress test of edge cases that a real cloud governance tool would encounter. A reviewer should understand the scenarios as a representative set, not as a comprehensive eval.

## Terraform stubs are hand-crafted to match scenarios.

The Terraform definitions used by the System Mapper are written alongside each scenario to match the scenario's sidecar metadata. They are not real-world Terraform — they are minimal stubs sufficient to exercise the System Mapper's parsing logic. The System Mapper's robustness to messy or unconventional real-world Terraform is not exercised.

---

## 10.4 What this is not

For completeness, the project is **not**:

- **Not a real-time monitoring system.** Reviews are bounded units of work, not stream processors.
- **Not a Terraform validator or linter.** Terraform is parsed to derive structure, not to check syntax or best-practice conformance.
- **Not a FinOps tool.** The recommendation set is narrower than a full FinOps platform would offer.
- **Not a security or compliance scanner.** Security posture is out of scope; compliance posture is out of scope.
- **Not a chatbot.** There is no conversational interface. The system runs as a pipeline; humans review at the end.

- **Not a multi-agent demo for demo's sake.** Every agent has a job that justifies its own layer — see `02-agents.md` §2.5.
- **Not a knowledge-graph project.** Knowledge representation is not the architectural focus. The audit trail is structured, but it is not a graph database in the semantic sense.

---

## 10.5 Where the limits would relax in a production-shaped extension

A reviewer asking "what would it take to make this production-ready?" gets a structured answer rather than hand-waving:

- **Real telemetry integration.** Replace the synthesized dataset with a connector to CloudWatch / Datadog / etc. The Input Harness's data-quality validation expands.
- **Production observability.** Operational metrics flow into Prometheus / OpenTelemetry. The audit trail stays focused on reasoning provenance.
- **Continual monitoring triggers.** Implement the Supervisor's change-driven, schedule-driven, and anomaly-driven triggers.
- **HITL UI.** Build the review-packet rendering into an actual reviewer dashboard.
- **Multi-application reasoning.** Add a separate layer above the per-application reviewer to aggregate patterns across applications.
- **Closed-loop feedback.** Add a mechanism that ingests HITL decisions back into the system to tune confidence thresholds and recommendation patterns.
- **Action whitelist expansion.** Add specialists and recommendations for security posture, IAM, encryption, data lifecycle.
- **Provider integrations for actuation.** If the system's identity shifts toward hybrid recommender-actuator, add bounded actuators behind the Action Harness.

Each of these is a real engineering project in its own right. Acknowledging the path from this portfolio implementation to a production system, without pretending the implementation already covers it, is what makes the scoping honest.

---

## 10.6 The point of this document

A senior reviewer evaluating a portfolio project notices both what the project does and what it claims. Claims that exceed what the project actually does are a red flag. Claims that match the project's actual scope, and that explicitly name the limits, are a green flag.

This document exists to make the green flag visible. Every architectural choice in this project was made within a particular scope; this document defines that scope. Anything beyond it is either out of scope (and named here) or future work (and named in `99-todo.md`).

## 12. Build Plan

This document sequences the work of turning the v1 design into a working portfolio implementation. It is a build plan, not a project plan — it focuses on the order of construction and the deliverables of each phase, not on day-by-day task management.

The plan assumes the synthetic dataset (Cloud Governance Dataset Generation Plan v3) is already specified and ready to be generated. Dataset generation is a parallel track that should produce all 18 scenarios before the agent system needs them — week 1 is a reasonable window for that work.

### Phase structure

The build is sequenced in four phases, each one week. The total window matches the original Project 2 build window (~four weeks). The phases are not rigid; some overlap between adjacent phases is expected.

| Phase   | Window | Focus                                                            |
|---------|--------|------------------------------------------------------------------|
| Phase 1 | Week 1 | Persistent Action Record + Input Harness + dataset generation    |
| Phase 2 | Week 2 | System Mapper + Supervisor + scenario plumbing                   |
| Phase 3 | Week 3 | Tier Specialists + Reasoning Harness                             |
| Phase 4 | Week 4 | Cross-Tier Evaluator + Action Harness + eval + README + LinkedIn |

The ordering reflects a deliberate principle: **build the audit trail first**. Every agent built afterward writes to the audit trail from day one, rather than having logging retrofitted onto a working system. This is the same principle the original Project 2 used and it is the right one. An audit trail that was built after the agents will always be incomplete in ways the agents do not notice; an audit trail that was built first forces every agent to be observable from its first commit.

### Phase 1 — Audit trail and ingest (Week 1)

**Goal:** A working audit trail that any subsequent component can write to, and a working Input Harness that validates a bundle before any agent runs.

#### Deliverables

- **Persistent Action Record storage layer.** SQLite database with the conceptual schema from [docs/06-persistent-action-record.md](#) §6.5 made concrete. Append-only enforcement. Foreign-key relationships in place. Indexes for the expected query patterns.

- **Audit trail write API.** A small Python library that each agent will use to log records. The library enforces append-only semantics and required fields. Records are not free-form; they conform to per-category schemas.
- **Audit trail read API.** A complementary library for replay tools and the eventual review-packet renderer. Includes the foreign-key traversal helpers that make "walk back from a recommendation to its evidence" a one-line call.
- **Input Harness.** Validates bundles against the dataset's v3 schema (telemetry tier arrays, sidecar fields, timestamps, Terraform parseability). Logs validation outcomes to the audit trail.
- **Dataset generation.** All 18 scenarios generated through Pass 1 and Pass 2, validated by the dataset's own QA (§8 of the dataset doc). This work runs in parallel with the audit-trail work; it should not block Phase 1 completion if the dataset pipeline is already operational.

## Why first

The audit trail is the system's primary observability mechanism. Building it first means every subsequent commit is observable from its first run. Building the Input Harness alongside it means downstream agents never have to handle malformed bundles, which keeps their failure modes clean.

## Exit criteria

A test bundle can be ingested through the Input Harness, validation outcomes are logged to the audit trail, and the read API can return the validation chain for inspection.

---

## Phase 2 — Orchestration and structure (Week 2)

**Goal:** The System Mapper and Supervisor are operational, and scenarios flow through the pipeline up to (but not including) specialist analysis.

## Deliverables

- **System Mapper.** Parses Terraform stubs (using `python-hcl2` or equivalent), classifies resources into tiers, builds the dependency graph, and produces the analysis plan. Cross-references the sidecar for configuration values. Logs the architecture model and analysis plan to the audit trail.
- **Terraform stubs for all 18 scenarios.** Hand-crafted to match each scenario's sidecar metadata. Minimal but parseable.
- **Supervisor.** Receives validated bundles and architecture models, decides which specialists to invoke, prepares the invocation manifest. Logs every decision. For Phase 2, specialist invocations are stubbed — the Supervisor produces the invocation commands but does not yet call real specialists.
- **End-to-end scenario plumbing.** A scenario flows from ingest, through the Input Harness, through the System Mapper, through the Supervisor's decision-making, and produces audit-trail records at each step. Specialists are stubbed; the rest is real.

## Why second

Orchestration and structure must be in place before agents can be meaningfully tested. Building the specialists before knowing how they are invoked, with what scope, against what tools, leads to specialists that are wrong-shaped for the orchestration that eventually wraps them.

## Exit criteria

Running any of the 18 scenarios through the pipeline produces a complete audit trail up through the Supervisor's specialist-invocation decisions, with the System Mapper's architecture model and analysis plan in place.

---

## Phase 3 — Reasoning depth (Week 3)

**Goal:** The three Tier Specialists are operational, with ReAct loops running against the read surface, producing structured findings that the Reasoning Harness validates.

## Deliverables

- **The ReAct tool surface.** Implementation of the operations defined in `docs/04-react-tool-surface.md`, per-agent scope enforcement, audit-trail logging of every tool call. Each tool call is a structured event in the audit trail.
- **Reasoning Harness.** Structured-output schema enforcement (the specialist finding schema from `docs/03-harnesses.md` §3.3). Evidence-binding required for any `issue_found` finding. Three-valued `finding_type` enum honored. Specialist-level confidence computed from named sub-signals.
- **Compute Analyst.** Operational against the compute tier's tool surface. Runs ReAct against the dataset's compute scenarios. Produces well-formed structured findings.
- **Data Layer Analyst.** Operational against the database and cache tiers. Same pattern.
- **Network Analyst.** Operational against the network tier. Same pattern.
- **Specialist eval against the 18 scenarios.** For each scenario, each invoked specialist produces a finding. Compare against the per-tier expectation derived from the target recommendation. Iterate prompts and confidence thresholds until findings match expectations.

## Why third

Specialists are the bulk of the system's intelligence. Building them after the orchestration scaffolding means they can be tested in their actual operating context (real audit trail, real Supervisor invocation), rather than in isolation.

## Exit criteria

For all 18 scenarios, the appropriate specialists run, the audit trail records their ReAct loops, and their findings match the per-tier expectations defined in the scenarios. Restraint scenarios (6, 16, 18) produce `no_issue_found` or low-confidence-minor-issue outputs. The `insufficient_data` path is exercised by at least one scenario in testing.

## Phase 4 — Synthesis, gating, and release (Week 4)

**Goal:** End-to-end system operational. Cross-Tier Evaluator synthesizes, Action Harness gates, evaluation methodology produces scored reports, and the repository is ready for public release.

### Deliverables

- **Cross-Tier Evaluator.** Drift-check on each specialist finding, cross-tier interaction identification, balanced recommendation synthesis with trade-off scoring, evaluator-level confidence. Uses Claude Sonnet. Logs every sub-step to the audit trail.
- **Action Harness.** Recommendation gate at the boundary between Evaluator and HITL. Well-formedness, evidence completeness, severity classification, duplication check. Logs gate verdict.
- **Review packet renderer.** Produces a structured review packet (the data contract specified in `docs/02-agents.md` and `docs/03-harnesses.md`). For Phase 4, the renderer produces JSON; the UI is deferred.
- **HITL stub.** For the portfolio demo, the HITL decision is captured via a CLI prompt or recorded manually. Real UI integration is a v2 deliverable. The HITL decision is logged to the audit trail regardless of mechanism.
- **Evaluation methodology implementation.** The five-rubric scorer from `docs/07-evaluation-methodology.md`. LLM-as-judge with Sonnet for R1–R4. Structural checker for R5. Per-scenario reports and aggregate distributions.
- **The three subset modes.** Smoke (3 scenarios), coverage (6 scenarios), full (18 scenarios). Each is a separate make target or CLI command.
- **Scenario replay CLI.** Given a scenario ID and an audit-trail entry, walks the chain and renders the full reasoning history as readable output. Mentioned in `docs/06-persistent-action-record.md` §6.3 and `docs/99-todo.md`.
- **README polish.** The audit-trail walkthrough is the visual hero. One scenario chosen, run end-to-end, every record shown in causal order. This is the section a hiring manager reads first.
- **LinkedIn launch post.** A short post (~300–500 words) describing the project, the architectural framing (four agents, four harnesses, audit-trail-as-hero), and a link to the repository. Should match the README's framing tightly.

### Why fourth

The Evaluator and Action Harness cannot be meaningfully built without specialists to consume. The eval cannot run without the full system. The README cannot be polished without the demo working. These dependencies fix the order.

### Exit criteria

The system runs end-to-end on all 18 scenarios. The full eval produces a structured report with per-scenario and aggregate scoring. The README contains the audit-trail walkthrough. The repository is in a state where it can be made public.



## Cross-cutting concerns through all phases

### Audit trail discipline

Every commit that adds an agent or a harness adds the corresponding audit-trail writes. There are no commits that "we'll add logging later." This is the principle that justifies building the audit trail first; it must be honored throughout.

### Documentation alongside code

Each phase produces or updates documentation in the `docs/` directory. The design documents in v1 are not contracts that the implementation must match exactly; they are the starting point, and they should be revised as implementation reveals what does and doesn't work. Significant deviations from the v1 design should produce documentation updates, not silent drift.

### Iteration on prompts

Specialist prompts, the Evaluator's drift-check prompt, and the LLM-as-judge eval prompt all need iteration. The right cadence is to tune them per scenario in Phase 3 (specialists) and Phase 4 (Evaluator and eval), running the affected subset eval after each significant prompt change.

### Cost monitoring

Each phase should produce a cost estimate for a representative run. By Phase 4, the cost-per-scenario and cost-per-full-eval should be known and documented in `docs/08-llm-and-cost.md` §8.3. The estimates in v1 of that document are placeholders; Phase 4 fills in the measured values.

---

## What ships at the end of Phase 4

A public GitHub repository containing:

- The agent system source code, structured along the design documents.
- The 18-scenario dataset (or scripts to regenerate it).
- The Terraform stubs for each scenario.
- The audit trail SQLite database schema and migration scripts.
- A `make demo` entry point that runs one scenario end-to-end.
- A `make eval` entry point that runs the full eval and produces a structured report.
- The full set of design documents updated to reflect what was actually built.
- A README whose audit-trail walkthrough makes the project understandable in five minutes.

The LinkedIn launch post goes up on the day of release, linking the repository.

---

## What does *not* ship at the end of Phase 4

- A UI for the HITL surfacing. The data contract is specified; the rendering is v2 work — see `docs/99-todo.md`.

- Multi-application reasoning, agent memory, closed-loop feedback, real telemetry integration. Out of scope — see [docs/10-limitations.md](#).
- A production deployment. The portfolio implementation runs locally.

These are deliberate non-deliverables. Naming them is part of the scoping discipline.